

COORDENADORIA DE ENGENHARIA DE COMPUTAÇÃO

FELIPE PIRES DOS SANTOS

SANDRO REINALDO GUARIGLIA FILHO

SOFTWARE DE CODIFICAÇÃO PARA ANÁLISES QUALITATIVAS

SOROCABA, SP

2024

FELIPE PIRES DOS SANTOS

SANDRO REINALDO GUARIGLIA FILHO

SOFTWARE DE CODIFICAÇÃO PARA ANÁLISES QUALITATIVAS

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do diploma de graduação em Engenharia de Computação.

Orientador (a): Dra. Andréia Damasio de Leles.

SOROCABA, SP

2024

FICHA CATALOGRÁFICA ELABORADA PELA "BIBLIOTECA FACENS"

Bibliotecária responsável

Eliane da Rocha CRB 8062/8a

S237s

Santos, Felipe Pires dos.

Software de codificação para análises qualitativas / por Felipe Pires dos Santos, Sandro Reinaldo Guariglia Filho. - Sorocaba, SP: [s.n.], 2024.

89f.

Trabalho de Conclusão de Curso (Graduação ou Pós) – Centro Universitário Facens – Coordenadoria de Engenharia da Computação – Curso de Engenharia da Computação, 2024.

Orientadora: Profa. Dra. Andreia Leles

1. Análise qualitativa. 2. Desenvolvimento de Software. 3. Codificação de dados qualitativos. I. Santos, Felipe Pires dos. II. Guariglia, Sandro Reinaldo. III. Centro Universitário Facens. IV. Título.

CDD 621.39

FELIPE PIRES DOS SANTOS

SANDRO REINALDO GUARIGLIA FILHO

SOFTWARE DE CODIFICAÇÃO PARA ANÁLISES QUALITATIVAS

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do certificado de conclusão do curso de Engenharia.

Orientador (a): Dra. Andréia Damasio de Leles.

BANCA EXAMINADORA

Prof. Dra. Andréia Damasio de Leles

Prof. Esp. Marco Antonio Montebello Junior

Prof. Esp. Fernando Tavares

AGRADECIMENTOS

Gostaríamos de expressar nossa profunda gratidão a todos os colegas e amigos que fizemos ao longo dessa jornada acadêmica, cujas trocas de conhecimento e companheirismo foram essenciais para a realização deste trabalho. Em especial, nossos sinceros agradecimentos aos companheiros do projeto ROTA 2030, desenvolvido nesta instituição, que nos ofereceram apoio, incentivo e confiança durante todo o processo, sendo parte fundamental no nosso crescimento pessoal e profissional.

Não poderíamos deixar de reconhecer com imensa gratidão o papel dos nossos professores, que ao longo do curso nos proporcionaram os conhecimentos necessários para desenvolvermos nosso potencial. Em especial, nosso sincero agradecimento à Profa. Dra. Andréia Damasio de Leles, nossa orientadora, seu apoio constante e suas orientações precisas foram determinantes para que pudéssemos superar os desafios que surgiram ao longo deste percurso.

Também dedicamos nossos agradecimentos ao grupo de pesquisa UXLeris, da Universidade Federal de São Carlos (UFSCar), sob a coordenação da professora Dra. Luciana Zaina. O grupo, além de nos proporcionar valiosas contribuições técnicas, serviu de inspiração pela sua excelência acadêmica e dedicação à pesquisa. A generosidade de tempo e esforço demonstrada pela equipe, especialmente no auxílio à avaliação deste projeto, foi essencial para o sucesso desta empreitada.

“O insucesso é apenas uma oportunidade para recomeçar com mais inteligência.”

Henry Ford

RESUMO

O presente trabalho aborda o desenvolvimento de uma plataforma voltada para facilitar a análise qualitativa de dados, com foco na etapa de codificação. O problema identificado é a limitação das ferramentas atuais, que são frequentemente caras ou complexas, o que restringe o acesso a pesquisadores com menos recursos ou familiaridade técnica. Nesse contexto, o objetivo principal é criar uma solução acessível que permita a codificação de dados textuais, organizando informações e auxiliando na extração de *insights*. A metodologia adotada foi baseada no ciclo de vida de desenvolvimento de software em modelo iterativo, que possibilitou ajustes contínuos durante o processo de criação. A implementação utilizou a linguagem Python e uma arquitetura de três camadas, garantindo flexibilidade e escalabilidade ao sistema. O software foi avaliado por meio de um estudo de caso com usuários reais, envolvendo testes de usabilidade e questionários de satisfação. Os resultados mostraram que a ferramenta é capaz de cumprir com as funcionalidades básicas que se propõe, mas ainda carece de implementações mais robustas, a fim de se tornar uma alternativa viável e acessível às soluções comerciais existentes.

Palavras-chave: Análise qualitativa. Desenvolvimento de Software. Codificação de dados qualitativos.

ABSTRACT

The present work addresses the development of a platform designed to facilitate qualitative data analysis, with a focus on coding techniques. The identified problem is the limitation of current tools, which are often expensive or complex, restricting access for researchers with fewer resources or limited technical expertise. In this context, the main objective is to create an accessible solution that allows the coding of textual data, organizing information and assisting in the extraction of insights. The methodology adopted was based on the software development life cycle in an iterative model, allowing for continuous adjustments throughout the creation process. The implementation was carried out using the Python language and a three-layer architecture, ensuring the system's flexibility and scalability. The software was evaluated through a case study with real users, involving usability tests and satisfaction questionnaires. The results showed that the tool can fulfill its basic functionalities, but still requires more robust implementations to become a viable and accessible alternative to existing commercial solutions.

Keywords: Qualitative analysis. Software Development. Qualitative data coding.

LISTA DE ILUSTRAÇÕES

FIGURA 1. Elementos de uma pesquisa de Natureza Qualitativa 18
FIGURA 2. Fluxograma *grounded theory* 20
FIGURA 3. Fluxograma da codificação baseada na *grounded theory* 23
FIGURA 4. Ciclo iterativo de vida de um software 26
FIGURA 5. Arquitetura em três camadas 28
FIGURA 6. Escala NPS 32
FIGURA 7. Casos de uso 40
FIGURA 8. Diagrama sequencial para abrir projeto existente 42
FIGURA 9. Diagrama de classes principal 43
FIGURA 10. Diagrama de componente para importador de textos 44
FIGURA 11. Tela inicial 45
FIGURA 12. Tela de codificação 46
FIGURA 13. Tela de gerenciamento de *tags* 47
FIGURA 14. Implementalação da classe "*Tag*" 48
FIGURA 15. Importadores e factory method 49
FIGURA 16. *Pipeline* de desenvolvimento e integração 52
FIGURA 17. Resultados questões 3,4, 7 e 10 55
FIGURA 18. Resultado questões 5 e 8 56
FIGURA 19. Resultados questão 2 57
FIGURA 20. Resultados questão 6 58
FIGURA 21. Resultados questões 1 e 9 58
FIGURA 22. Respostas para o NPS 60
FIGURA 23. Exemplo de codificação 61
FIGURA 24. Nuvem de palavras relacionadas a "Aprendizado" 62
FIGURA 25. Nuvem de palavras relacionadas a "Funcionalidades" 62
FIGURA 26. Nuvem de palavras relacionadas a "*layout*" 63
FIGURA 27. Notas e adjetivos associadas aos escores brutos do SUS 64
FIGURA 28. Tela de abertura do software 85
FIGURA 29. Aba “projeto” do software 86
FIGURA 30. Janela de adicionar questão 86

FIGURA 31. Aba “*tags”* do software 87
FIGURA 32. Janela de criação de *tags* 87
FIGURA 33. Procedimento para codificação 88
FIGURA 34. Opções de salvar e exportar o projeto 88

LISTA DE SIGLAS

CAQDAS	*Computer-assisted qualitative data analysis software*
NPS	*Net Promoter Score*
QDA	*Qualitative data analysis*
SDLC	*Software Development Life Cycle*
SUS	*System Usability Scale*

SUMÁRIO

1 INTRODUÇÃO 14
2 PESQUISA E ANÁLISE EM DADOS QUALITATIVOS 17
2.1 DADOS QUALITATIVOS X QUANTITATIVOS 17
2.2 A NATUREZA DE PESQUISAS QUALITATIVAS 18
2.2.1 Estratégias, paradigmas e abordagens 19
2.2.2 Coleta de dados qualitativos 20
2.3 PROCESSAMENTO E ANÁLISE DE DADOS QUALITATIVOS 21
2.3.1 Codificação dos dados 21
2.3.1.1 Técnicas e processo de codificação 22
2.4 ANÁLISES PÓS-CODIFICAÇÃO 23
3 METODOLOGIA 25
3.1 CICLO DE VIDA DE UM SOFTWARE 25
3.2 A ARQUITETURA DE TRÊS CAMADAS 27
3.3 AVALIAÇÃO 29
3.3.1 Metodologia de avaliação 29
3.3.2 Métricas de sucesso 30
3.3.3 Coleta dos dados 31
4 DESENVOLVIMENTO 34
4.1 FASE DE PLANEJAMENTO E ANÁLISE 34
4.1.1 Análise de viabilidade técnica 34
4.1.1.1 Análise de mercado 34
4.1.1.2 Análise de produtos 36
4.1.1.3 Viabilidade e esforços de desenvolvimento 37
4.1.2 Definição do contexto do sistema 38
4.1.3 Definição dos requisitos 39
4.2 FASE DE *DESIGN* 40
4.2.1 Definição dos casos de uso 40
4.2.2 Especificação dos componentes 42
4.2.3 Interface gráfica do usuário 44
4.3 FASE DE IMPLEMENTAÇÃO 47
4.3.1 Implementação dos componentes 47

4.3.2 Criação de testes unitários 50
4.3.3 Desenvolvimento e integração contínua 51
4.3.4 Documentação dos componentes 52
5 RESULTADOS 54
5.1 RESULTADOS QUANTITATIVOS 54
5.1.1 Resultados *System Usability Scale* 54
5.1.1.1 Facilidade de aprendizagem 55
5.1.1.2 Eficiência 56
5.1.1.3 Facilidade de memorização 56
5.1.1.4 Minimização dos erros 57
5.1.1.5 Satisfação 57
5.1.2 Resultados *Net Promoter Score* 59
5.2 RESULTADOS QUALITATIVOS 60
6 DISCUSSÃO 64
6.1 O SOFTWARE NA ESCALA SUS E NPS 64
6.2 AS RESPOSTAS QUALITATIVAS 66
6.3 LIMITAÇÕES 67
7 CONSIDERAÇÕES FINAIS 69
REFERÊNCIAS 71
APÊNDICE A – *PRODUCT VISION BOARD* 76
APÊNDICE B – *STAKEHOLDERS* IDENTIFICADOS 77
APÊNDICE C – REQUISITOS DO SISTEMA 79
APÊNDICE D – CASOS DE USO 82
APÊNDICE E – APRESENTAÇÃO E TUTORIAL DE USO DO SOFTWARE 85
ANEXO A - QUESTIONÁRIO *SYSTEM USABILITY SCALE* 89

1 INTRODUÇÃO

No mundo moderno, observa-se um aumento exponencial no volume de dados, impulsionado pela proliferação da tecnologia digital e da conectividade global (Vianna, 2021). Este influxo massivo de informações abrange uma ampla variedade de tipos, desde dados estruturados até não estruturados, tanto quantitativos quanto qualitativos. Tais dados demandam processos de gestão cuidadosos, que envolvem desde a coleta até a forma como são analisados e armazenados (Garg; Popli; Sarao, 2021).

Embora os dados quantitativos sejam frequentemente valorizados por sua objetividade e capacidade de análise estatística direta, os dados qualitativos desempenham um papel igualmente crucial, fornecendo *insights* ricos e nuances sobre aspectos subjetivos e contextuais da experiência humana. Desde análises de sentimentos em mídias sociais (Sforza; dos Santos, 2023) até pesquisas qualitativas em ciências sociais (Goldenberg, 2003), os dados qualitativos oferecem uma perspectiva valiosa que complementa e enriquece a compreensão dos fenômenos complexos que permeiam a sociedade.

Em vista disso, desenvolveram-se diversas técnicas para analisar essas informações de caráter subjetivo, como o processo de codificação para a análise semântica, associando códigos à trechos específicos do conteúdo (Ho; Limpaecher, 2022). Paralelamente, ferramentas que auxiliam nessa atividade foram desenvolvidas, os chamados *Computer-assisted qualitative data analysis software* (CAQDAS) possuem recursos que agilizam esse processo. Todavia, a maioria dos CAQDAS são caracterizadas por custos substanciais e uma estrutura de código fechado, como o MAXQDA, ATLAS.ti, NVivo e Delve, o que acarreta a limitação do acesso democrático a tais recursos no âmbito da pesquisa.

Por outro lado, alternativas gratuitas de código aberto, como o projeto QualCoder (Curtain, 2023), apresentam desafios próprios, como a escassez de documentação e uma organização do código menos sistemática. Outro exemplo é o projeto Label Studio (Tkachenko *et al.*, 2022), voltado para rotulação de dados para aprendizado de máquina, que oferece recursos específicos para essa técnica, mas não atende completamente às necessidades de pesquisa qualitativa.

Desse modo, o presente trabalho busca por desenvolver um CAQDAS destinado a otimizar o processo de análise qualitativa, com foco primário na facilitação da etapa de codificação dos dados. Para atingir este objetivo, o projeto desdobra-se em metas específicas. Em primeiro lugar, é a criação de uma plataforma capaz de importar diversos formatos de textos, oferecendo flexibilidade na manipulação de dados. Em seguida, o software será dotado de ferramentas para a criação de rótulos, ou *tags*, permitindo aos pesquisadores associá-los a trechos do conteúdo analisado. Além disso, buscou-se oferecer uma análise simplificada e uma visão geral do progresso da codificação, proporcionando aos usuários uma compreensão clara e abrangente do processo em andamento.

Esses objetivos convergem para a criação de uma ferramenta versátil, projetada para aprimorar a eficiência da análise qualitativa de dados. O caráter público e colaborativo do projeto desempenha um papel importante na redução da escassez de recursos desse tipo na internet, ao mesmo tempo em que possibilita uma evolução contínua do software por meio de sugestões e colaborações da comunidade. Dessa forma, além de oferecer uma ferramenta de baixo custo, o projeto também serve como um recurso de apoio à pesquisa e análise de dados, contribuindo para o avanço do conhecimento e da prática científica.

O desenvolvimento do software, portanto, foi conduzido na linguagem Python, escolhida por sua flexibilidade e ampla gama de recursos que podem ser aproveitados para atender às necessidades específicas do projeto. Toda a estrutura e lógica de negócios serão elaboradas seguindo os padrões e estilos de código estabelecidos pela comunidade Python, garantindo assim uma base sólida e coesa para o desenvolvimento da ferramenta. Além disso, o projeto adotará boas práticas de engenharia de software, com destaque para a utilização de um ciclo de desenvolvimento iterativo e uma arquitetura de três camadas.

Com um foco especial na qualidade e na robustez do software, será adotado práticas de programação e testagem. Isso incluirá a realização de testes automatizados abrangentes em todas as etapas do desenvolvimento, garantindo que o software seja estável e ofereça uma boa experiência de uso. Além disso, a documentação detalhada será uma adição, fornecendo informações claras e concisas sobre a instalação, configuração e uso do software, a fim de facilitar a adoção e a compreensão por parte dos usuários, bem como possibilitar contribuições e

colaborações externas.

Por meio dessa abordagem técnica, esperou-se que o software não apenas atenda às expectativas dos usuários, mas também estabelecesse um padrão de qualidade e confiabilidade no campo da análise qualitativa de dados. Assim, os resultados foram verificados e validados através de testes de usabilidade. Essas análises permitirão uma avaliação abrangente da ferramenta, identificando suas vantagens e áreas de melhoria de forma conclusiva, com o intuito de garantir sua relevância e utilidade para os usuários.

Assim, o documento terá a seguinte estrutura: o capítulo 2 abordará a questão teórica da pesquisa e análise em dados qualitativos, explicando as principais técnicas. O capítulo 3 terá uma contextualização sobre a metodologia aplicada no desenvolvimento do produto, seguido pela seção 4 onde descreverá todas as fases do desenvolvimento. Por fim, os resultados e uma discussão acerca serão apresentadas nas seções 5 e 6, finalizando com as considerações finais do trabalho.

2 PESQUISA E ANÁLISE EM DADOS QUALITATIVOS

A pesquisa qualitativa, ou descritiva, desempenha um papel importante na compreensão e exploração da complexidade dos fenômenos sociais, culturais e psicológicos (Allan; Skinner, 2020; Olsen, 2015; Rahman, 2016). Nesse contexto, a análise de dados qualitativos, do inglês – *Qualitative data analysis* (QDA), emerge como uma abordagem para extrair *insights* significativos a partir de dados ricos e contextualizados.

2.1 DADOS QUALITATIVOS X QUANTITATIVOS

Para uma compreensão mais aprofundada do processo de QDA, é necessário entender e discernir entre os tipos de dados e os métodos de pesquisa associados a eles. Enquanto a pesquisa quantitativa busca quantificar e medir fenômenos por meio de números e estatísticas, a pesquisa descritiva se concentra na compreensão em profundidade dos significados, experiências e perspectivas dos participantes (Rahman, 2016).

Dessa forma, a diferença entre os tipos de dados presentes em cada uma pode ser definida de acordo com a seguinte afirmação:

> Os dados quantitativos são condensados, distintos e discretos e podem ser representados como números em uma tabela ou matriz contendo variáveis não sobrepostas e seus valores. Por outro lado, os dados qualitativos não têm uma estrutura de tabela ou lista e não consistem em variáveis não sobrepostas e seus valores. Os dados qualitativos podem ter várias estruturas, por exemplo, a estrutura de um texto, com palavras formando frases. Os dados qualitativos podem ser codificados, mas esses códigos podem se sobrepor em termos de posição e/ou significado. (Schoonenboom, 2023, p. 18, tradução nossa).

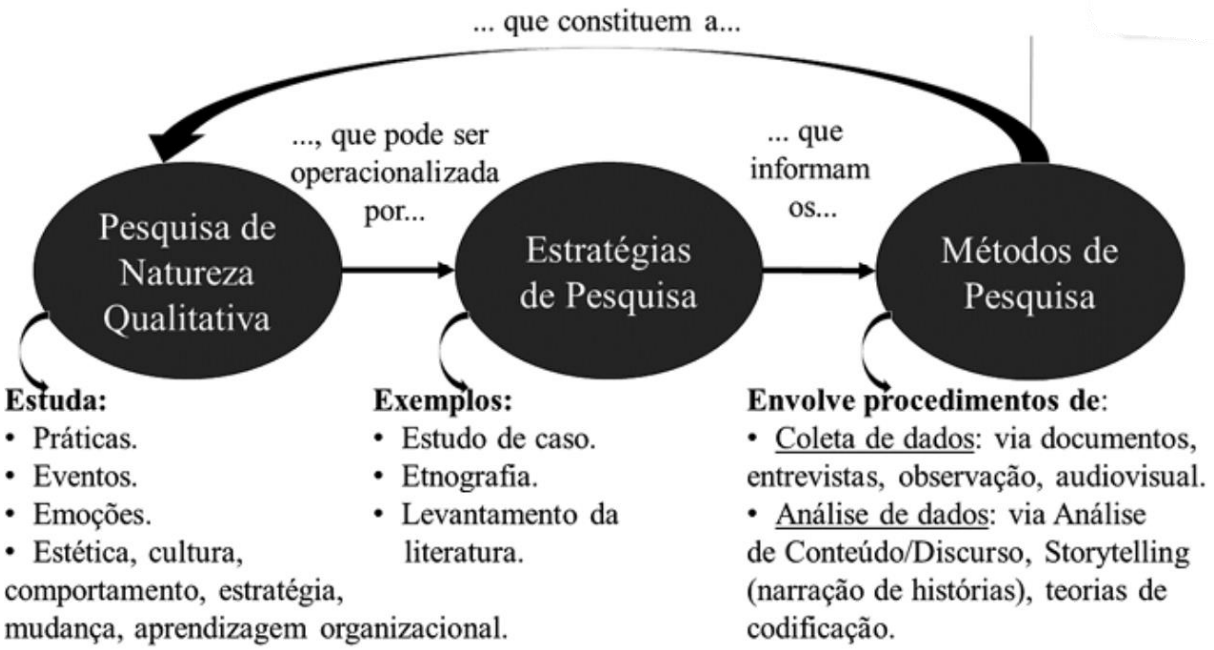
A partir de então, as metodologias de coleta e análise se diferenciam significativamente entre ambos os tipos. Enquanto para o quantitativo existem técnicas robustas como sondagens, questionários e experimentos controlados (Rahman, 2016), o qualitativo se apoia em métodos mais flexíveis e exploratórios, tais como entrevistas, observações de participantes e análise de conteúdo (Olsen, 2015). Essas abordagens distintas refletem as diferentes ênfases na pesquisa, com a quantitativa buscando dados mensuráveis para análise estatística, e a qualitativa buscando compreensão profunda e contextualizada dos fenômenos em estudo.

2.2 A NATUREZA DE PESQUISAS QUALITATIVAS

Dada a distinção entre os tipos de pesquisa, tanto na sua natureza básica quanto aplicada, quando assumem uma abordagem descritiva, estas se tornam empíricas e investigativas (Mattar; Ramos, 2021), proporcionando uma interpretação tangível a partir de elementos etnográficos e até mesmo subjetivos coletados (Aspers; Corte, 2019).

Esse tipo de investigação pode ser então operacionalizado conforme a estrutura definida por Américo (2021), onde as estratégias são definidas caráter da pesquisa e informam os métodos para a condução do trabalho, conforme apresenta o diagrama da Figura 1.

Figura 1 - Elementos de uma pesquisa de Natureza Qualitativa



Fonte: (Américo, 2021, p. 18)

A pesquisa qualitativa, portanto, tem como objetivo compreender os fenômenos comportamentais, as emoções e os eventos humanos através de estratégias e paradigmas que orientam o pesquisador a adotar abordagens holísticas. Essas abordagens demandam métodos específicos para a coleta e análise de cada tipo de informação, possibilitando uma compreensão mais profunda e contextualizada desses aspectos.

2.2.1 Estratégias, paradigmas e abordagens

Ao adentrar nesse campo, compreender as diretrizes que orientam o processo de investigação. Estas diretrizes metodológicas não apenas delineiam o caminho a seguir, mas também moldam a maneira como os pesquisadores concebem e interpretam os fenômenos estudados, também podendo ser chamadas de paradigmas de pesquisa:

> O paradigma funcionaria como o mapa ou roteiro de uma ciência, fornecendo critérios para delimitar problemas, formular questões, determinar o que deve ser observado, coletar dados, analisar resultados e assim por diante. Corresponderia, portanto, a um conjunto de práticas que caracterizam determinada ciência, incorporando os valores e as crenças dos cientistas da área. (Mattar; Ramos, 2021, p. 23).

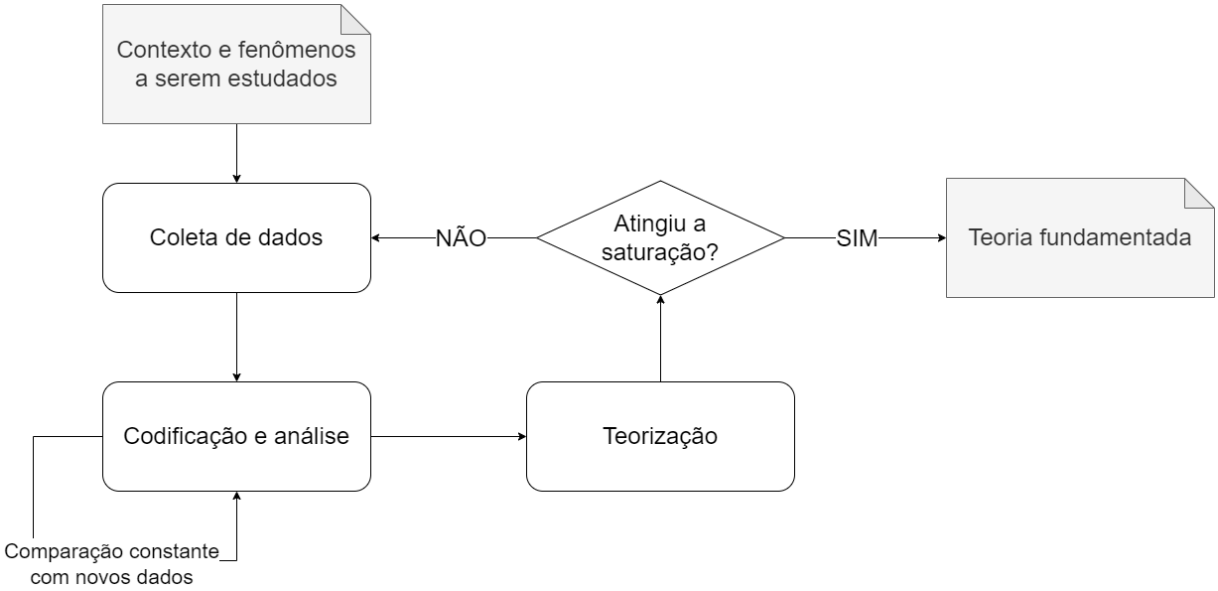
Assim, no âmbito qualitativo, os paradigmas mais comuns utilizados para guiar os estudos estão relacionados com a fenomenologia (Groenewald, 2004), etnografia (Sangasubana, 2014) ou a *grounded theory* (Corbin; Strauss, 2015).

A fenomenologia é uma abordagem filosófica e método de pesquisa qualitativa que busca compreender os fenômenos humanos a partir da perspectiva dos próprios sujeitos envolvidos (Qutoshi, 2018). Em pesquisas qualitativas, a fenomenologia procura investigar os significados que as pessoas atribuem às suas experiências incluindo seus pensamentos, sentimentos e percepções (Bliss, 2016).

Semelhantemente, a etnografia se concentra na compreensão das práticas culturais, valores, crenças e comportamentos de um grupo específico de pessoas em seu contexto natural (Ayar; Yalvac, 2022; Sangasubana, 2014). Diferentemente da fenomenologia, que se concentra nas experiências individuais, a etnografia busca compreender as dinâmicas culturais de um grupo social em particular.

A teoria fundamentada, ou *grounded theory*, por outro lado traz uma abordagem mais indutiva. Este método visa desenvolver teorias explicativas ou interpretativas a partir dos dados coletados durante a pesquisa, permitindo que as hipóteses surjam organicamente a partir das observações e interações dos participantes (Glaser; Strauss, 2017). Por seu caráter sistemático, a *grounded theory* segue etapas pré-definidas para chegar a uma teoria, conforme mostra o fluxograma da Figura 2.

Figura 2 - Fluxograma *grounded theory*



Fonte: elaborada pelos autores.

O paradigma inicia-se com a coleta de uma ampla gama de dados relevantes para o fenômeno estudado. Em seguida, os pesquisadores realizam a codificação dos dados, identificando padrões, temas e conceitos emergentes, este processo é iterativo, com uma comparação constante entre novos dados e dados existentes, refinando categorias e conceitos conforme necessário.

A partir da análise dos dados codificados, os pesquisadores desenvolvem teorias ou modelos explicativos que descrevem e explicam os padrões e processos observados, sendo essas teorias fundamentadas nos próprios dados, derivadas diretamente das observações feitas durante a pesquisa. Por fim, a saturação teórica é atingida quando os pesquisadores consideram que já coletaram dados suficientes e que novas informações não estão mais levando a insights significativos ou mudanças nas teorias desenvolvidas, indicando que a análise está suficientemente desenvolvida.

2.2.2 Coleta de dados qualitativos

Embora os paradigmas se diferenciem entre si, o ponto em comum entre eles são as metodologias de coleta de dados, que em todos os casos buscam capturar as experiências, perspectivas e significados dos participantes de uma pesquisa (Flick, 2018). Ademais, considerando a natureza humana do estudo, é imprescindível adotar um rigor ético em todas as fases da pesquisa (Olsen, 2015). Isso implica seguir as normativas estabelecidas por comitês de ética, assegurando a integridade e o bem-estar dos participantes durante todo o processo de investigação.

Dessa forma, os métodos de coleta são diversos, sendo os mais comuns para essas abordagens são:

Entrevistas: estruturadas, semiestruturadas ou abertas, as entrevistas permitem uma exploração detalhada das experiências e opiniões dos participantes. **Observação Participante**: envolve a imersão do pesquisador no ambiente natural dos participantes, permitindo uma compreensão profunda dos comportamentos e interações. **Grupos Focais**: reúnem um grupo de participantes para discussão de temas específicos, possibilitando a emergência de diferentes perspectivas e interações sociais. **Análise de Documentos**: exame de documentos escritos, como diários, cartas ou registros organizacionais, para entender contextos históricos, culturais ou institucionais.

Observa-se que em todos esses cenários, os dados gerados exibem um caráter desestruturado e subjetivo, refletindo-se em uma diversidade de formas, tais como textos, áudios ou outros artefatos. Essa heterogeneidade na sua apresentação demanda uma abordagem flexível e sensível na análise, considerando a riqueza e complexidade das experiências humanas (Américo, 2021).

2.3 PROCESSAMENTO E ANÁLISE DE DADOS QUALITATIVOS

Diante a ampla variedade de informações, é importante adotar abordagens meticulosas para extrair delas *insights* relevantes ou para desenvolver teorias robustas e fundamentadas (*grounded theory*).

Os dados qualitativos, pela sua própria natureza, frequentemente se manifestam, ou são transformados, em forma textual, o que possibilita uma QDA mais aprofundada e interpretativa (Lester; Cho; Lochmiller, 2020). Assim, para lidar com essa complexidade, emprega-se estratégias que permitam estruturar e compreender esses dados de maneira eficiente. Uma dessas estratégias é a utilização de técnicas de codificação, que permitem uma organização sistemática e uma análise mais criteriosa das informações.

2.3.1 Codificação dos dados

Sendo a técnica mais comum para o processamento dos dados subjetivos, a codificação qualitativa é uma das tarefas mais importantes na pesquisa qualitativa. Dessa forma podendo ser definida como “um processo sistemático, subjetivo e

transparente de redução de dados a conceitos significativos e confiáveis que representam adequadamente os dados e abordam o problema da pesquisa, objetivo ou pergunta” (Adu, 2019, p. 23).

Dessa forma, a codificação consiste na técnica de associar códigos, etiquetas, rótulos ou *tags* à determinados trechos da informação. Um código pode ser definido como “uma palavra ou frase curta que atribui simbolicamente um atributo sumativo, saliente, que captura a essência e/ou evocativo para uma parte dos dados” (Saldana, 2012, p. 3).

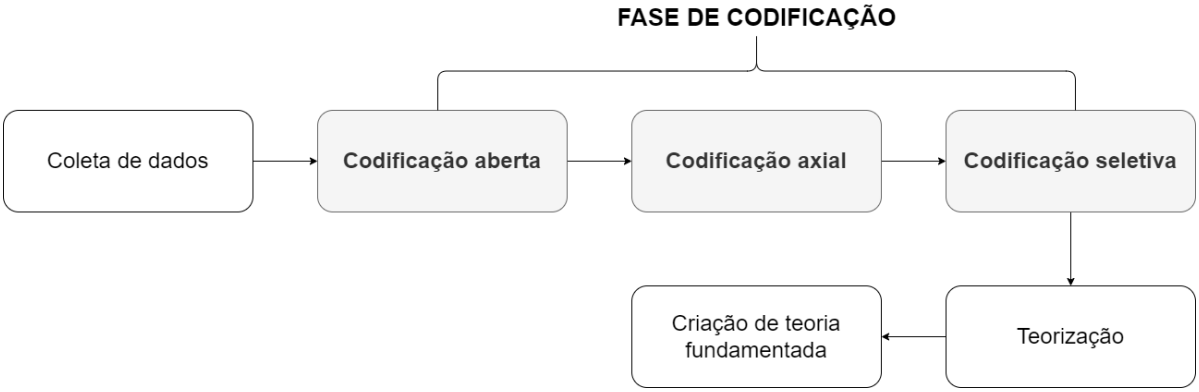
Embora essa abordagem seja uma ferramenta poderosa, também apresenta desafios e considerações importantes. É fundamental que os pesquisadores estejam conscientes de questões como vieses pessoais, interpretação subjetiva e contexto cultural, que podem influenciar o processo de codificação e os resultados obtidos (Bytheway, 2018). Além disso, à medida que o volume de informações cresce, o processo de codificação pode se tornar cada vez mais complexo e exigente para o pesquisador, demandando um esforço significativo para garantir uma análise detalhada e precisa (Mohajan; Mohajan, 2022).

2.3.1.1 Técnicas e processo de codificação

Dentro dessa metodologia ainda existem tipos de codificação que podem ser aplicadas em conjunto ou isoladamente. Williams; Moser (2019) definem que ao utilizar essa técnica, baseando-se na *grounded theory*, passa por 3 estágios de codificação: aberta, axial e seletiva, conforme mostra o fluxograma da Figura 3. O número de rótulos diminui constantemente ao longo dessas etapas.

Durante a primeira fase, codificação aberta, busca por dividir os dados qualitativos em partes discretas, examinando-os de perto e comparando-os quanto a semelhanças e diferenças (Corbin; Strauss, 2015). O pesquisador, portanto, começa a atribuir os códigos de acordo com a sua percepção. Posteriormente esses códigos serão agrupados e refinados de forma iterativa, conforme novos dados ou insights são descobertos e os estágios da codificação avançam.

Figura 3 - Fluxograma da codificação baseada na *grounded theory*



Fonte: adaptado de (Williams; Moser, 2019, p. 47)

Na fase subsequente da análise, conhecida como codificação axial, o pesquisador realiza uma refinada análise dos códigos previamente atribuídos durante a codificação aberta. Nesse estágio, o foco recai sobre a identificação das relações e conexões entre os códigos, buscando compreender como os diferentes elementos se interrelacionam e se agrupam em categorias mais amplas (Mohajan; Mohajan, 2022). O pesquisador examina de forma sistemática as associações entre os códigos, explorando as complexidades e padrões subjacentes aos dados.

No estágio final, codificação seletiva, o esforço concentra-se na seleção dos códigos mais significativos e relevantes para a investigação (Williams; Moser, 2019). Aqui, os códigos previamente identificados durante a codificação aberta e axial são cuidadosamente revisados e avaliados à luz dos objetivos da pesquisa e dos principais temas emergentes (Skjott Linneberg; Korsgaard, 2019). O pesquisador busca identificar os códigos que melhor representam os padrões e tendências mais proeminentes nos dados, priorizando aqueles que oferecem *insights* mais profundos e contribuem para uma compreensão abrangente do fenômeno em estudo (Saldana, 2012).

2.4 ANÁLISES PÓS-CODIFICAÇÃO

Embora envolvam processos diferentes, a codificação e análise posterior são intrinsicamente ligadas e interdependentes (Rogers, 2023). A qualidade das análises realizadas sobre os dados é crucial para assegurar a confiabilidade e a solidez da pesquisa (Skjott Linneberg; Korsgaard, 2019), através da transparência e comunicação adequada dos processos (Lester; Cho; Lochmiller, 2020).

Tendo em base os resultados da codificação, os dados agora estão mais

estruturados e menos subjetivos. A partir de então, o pesquisador é capaz de formular teorias indutivas ou dedutivas acerca do foco da pesquisa e fundamentada nos dados (Strauss; Corbin, 1998). Para isso, é de cargo do investigador utilizar-se de metodologias que avaliem os resultados obtidos da codificação.

Dentre as abordagens, os mais adotados envolvem uma análise do conteúdo, uma análise da temática ou até mesmo uma análise da narrativa, todos esses buscam por compreender a natureza dos dados qualitativos, extraindo informações que sejam valiosas para a resposta de perguntas delineadas no escopo do projeto.

Evidencia-se, portanto, que durante a pesquisa qualitativa, grande parte do esforço está por conta do próprio pesquisador, desde a coleta até a análise dos dados. Atualmente, existe softwares que auxiliam esse processo, realizando até mesmo análises e codificações automáticas em níveis mais genéricos (Kaufmann; Barcomb; Riehle, 2020). Todavia, quando a tarefa se torna específica e requer uma precisão maior, o trabalho ainda demanda o esforço de uma pessoa (Rogers, 2023).

Nesse sentido, os softwares especializados em análise qualitativa emergem como recursos valiosos para o pesquisador durante esse tipo de investigação, conferindo-lhe uma vantagem em termos de robustez e eficiência no processamento e análise dos dados (Kaufmann; Barcomb; Riehle, 2020; Rogers, 2023). Essas ferramentas oferecem uma variedade de funcionalidades projetadas especificamente para atender às complexas demandas da análise de dados qualitativos (Deterding; Waters, 2021).

3 METODOLOGIA

Dadas as características e desafios inerentes à pesquisa qualitativa, bem como das técnicas de codificação que sustentam as suas análises, é necessário delinear a metodologia que guiará o desenvolvimento de um software que atenda aos requisitos desse campo.

A seção que segue descreve a metodologia aplicada no desenvolvimento do software, desde a concepção da sua arquitetura até a avaliação. Esse processo metodológico será estruturado para garantir que o software não apenas reflita os princípios discutidos na seção anterior, mas também ofereça uma solução prática e adaptável às demandas reais de análise qualitativa.

3.1 CICLO DE VIDA DE UM SOFTWARE

O desenvolvimento de software, no presente contexto de uma aplicação voltada para a análise qualitativa de dados, exige um planejamento cuidadoso e estruturado. O ciclo de vida de um software – do inglês, *Software Development Life Cycle* (SDLC), é uma metodologia que descreve as etapas envolvidas desde a concepção inicial do projeto até a sua finalização e manutenção (Royce, 1987). Essa abordagem sistemática garante que a aplicação seja desenvolvida de forma eficiente, atendendo aos requisitos de qualidade e funcionalidades esperados.

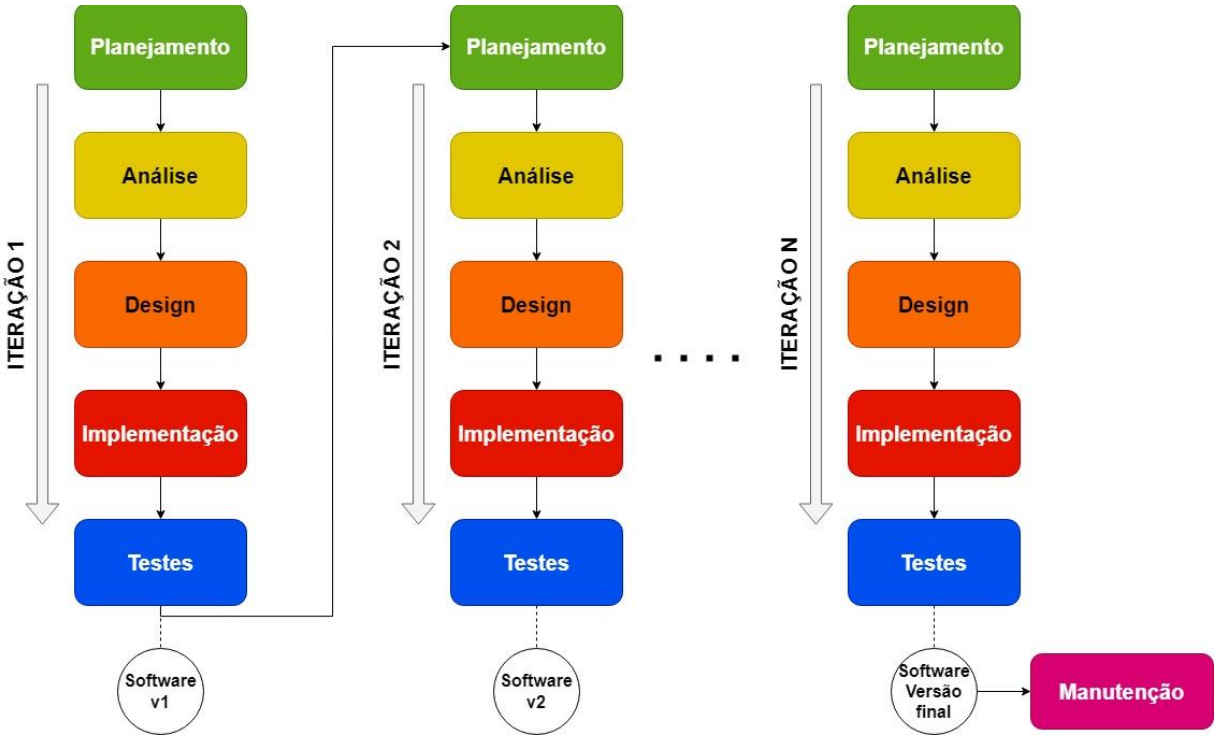
O SDLC pode ser dividido em várias etapas, cada uma com suas atividades específicas, objetivos e entregáveis. Entre os modelos mais conhecidos estão o modelo em cascata (*Waterfall*), o modelo incremental, o modelo espiral, e as metodologias ágeis (*Agile*) (Sommerville, 2019).

Neste projeto, adotou-se uma abordagem iterativa (Larman; Basili, 2003). A implementação dessa metodologia favorece o refinamento constante, com foco na entrega incremental de valor, seguindo um ciclo contínuo de avaliação e ajustes. Dessa forma, o processo de desenvolvimento pode seguir a seguinte colocação:

> O processo iterativo sugere que as equipes comecem o desenvolvimento de software com um pequeno subconjunto de requisitos. Com o passar do tempo, eles aprimoram as versões iterativamente até que o software completo esteja pronto para produção. A equipe produz uma nova versão do software ao final de cada iteração (AWS, 2024).

Portanto, ao término de cada iteração, uma nova versão do software será disponibilizada para avaliação, permitindo que ajustes sejam feitos de acordo com o *feedback* das partes interessadas. Esse ciclo contínuo de refinamento seguirá até que se atinja um consenso sobre a versão final, atendendo plenamente às expectativas de todos os envolvidos, conforme mostrado pela Figura 4.

Figura 4 - Ciclo iterativo de vida de um software



Fonte: adaptado de (Tutorialspoint, 2024).

A escolha dessa vertente do SDLC para o projeto foi motivada pela justamente pela capacidade de gerar uma nova versão a cada iteração. Esse processo de evolução e avaliação constante permite que riscos sejam identificados e gerenciados com maior facilidade, já que os requisitos podem ser ajustados entre as iterações (AWS, 2024). E isso ocorre sem a necessidade das cerimônias formais presentes em abordagens como a ágil.

Os estágios apresentados, portanto, conduzirão todo o desenvolvimento do presente projeto. Cada passo é responsável por um aspecto de qualidade do produto, e podem ser descritos da seguinte forma:

a) Fase de Planejamento:

A primeira etapa do ciclo de vida de um software é o planejamento. Nessa fase, são definidas as metas do projeto, escopo, cronograma e os recursos necessários. O

planejamento detalhado busca prever possíveis desafios e alocar adequadamente os recursos;

b) Fase de Análise de Requisitos:

A análise de requisitos é onde se detalham as necessidades dos usuários e as funcionalidades que o software deve oferecer. Envolve a coleta e documentação de requisitos, funcionais e não funcionais;

c) Fase de *Design*:

A fase de *design* é onde a arquitetura do software é definida. Neste projeto, optou-se por uma arquitetura de três camadas, composta pela camada de apresentação, camada de lógica de negócios e camada de dados;

d) Fase de Implementação:

Durante esse passo, o *design* é traduzido em código. O software foi desenvolvido utilizando a linguagem Python, devido à sua versatilidade e ampla adoção em projetos de análise de dados;

e) Fase de Testes:

A fase de testes é fundamental para garantir que o software funcione conforme o esperado e atenda aos requisitos estabelecidos. Podem ser implementados testes unitários e testes de integração ao longo do desenvolvimento para verificar a funcionalidade individual dos componentes e a interação entre eles;

f) Fase de Manutenção:

Após a implementação e o lançamento, o software em versão final, entra no estágio de manutenção. Esta etapa envolve a correção de possíveis erros identificados pelos usuários, a atualização do software para se adequar a novos requisitos e a melhoria contínua baseada no feedback dos usuários. A manutenção preza por garantir que o software permaneça funcional e relevante ao longo do tempo.

3.2 A ARQUITETURA DE TRÊS CAMADAS

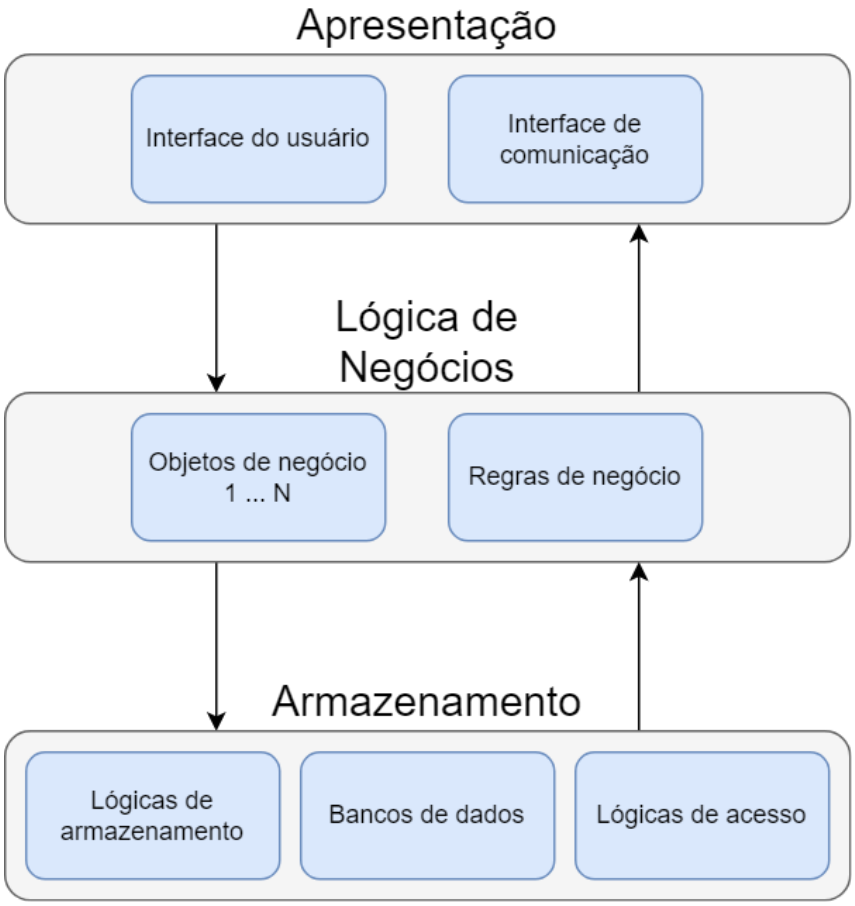
Conforme citado anteriormente, para o desenho do sistema apresentado, optou-se pela arquitetura em três camadas, sendo uma abordagem amplamente utilizada no desenvolvimento de sistemas de software, especialmente em aplicações

que demandam escalabilidade (Laplante; Kassab, 2022). As arquiteturas de N-*tiers* promovem uma manutenção facilitada e separação clara de responsabilidades (Gorton, 2011).

Essa construção divide o sistema em três camadas principais: a camada de apresentação, a camada de lógica de negócios e a camada de armazenamento de dados. Cada uma dessas camadas desempenha um papel específico no funcionamento do software, contribuindo para a modularidade e flexibilidade do sistema, podendo até ser migrado facilmente para outros tipos, como a arquitetura de microsserviços (Nisansala *et al.*, 2022).

A arquitetura, portanto, pode ser pensada conforme mostra a Figura 5, onde cada camada só se comunica com a imediata camada inferior, através de interfaces que realizam a troca das informações.

Figura 5 - Arquitetura em três camadas



Fonte: elaborada pelos autores.

A **camada de apresentação** é responsável pela interface com o usuário, ou seja, a parte do sistema com a qual o usuário interage diretamente. Ela transforma as

entradas do usuário em comandos para a camada de lógica de negócios e apresenta os resultados processados de forma intuitiva e acessível.

A **camada de lógica de negócios**, por sua vez, é o núcleo funcional do sistema. É responsável por implementar as regras de negócio e processar os dados que vêm da camada de apresentação antes de enviá-los à camada de armazenamento, ou vice-versa. Em um software de codificação para análises qualitativas, essa camada gerencia as operações essenciais, como a criação, edição e gerenciamento de códigos e rótulos, a associação desses códigos aos dados e a execução de análises sobre o material codificado (Gorton, 2011).

Por último, a **camada de armazenamento** é responsável pela persistência dos dados, ou seja, onde os dados são armazenados, organizados e gerenciados. Esta camada interage diretamente com a camada de lógica de negócios, recebendo instruções para armazenar, atualizar, ou recuperar informações conforme necessário (Gorton, 2011).

A integração entre cada camada é feita por interfaces, que permitem a troca das informações. Por exemplo, entre as camadas de apresentação e lógica de negócios, a transferência pode ser feita por um código que tem essa função, as chamadas classes adaptadoras (Gorton, 2011).

3.3 AVALIAÇÃO

Por fim, a avaliação do software gerado é uma etapa indispensável do SDLC escolhido, sendo possível identificar se os objetivos e requisitos estabelecidos foram devidamente cumpridos, quais são os pontos de melhorias, e se o sistema atende às expectativas dos usuários finais. Para avaliar o presente projeto, optou-se pela realização de um estudo de caso.

3.3.1 Metodologia de avaliação

Quanto à metodologia de estudo de caso, Lüdke e André, (2022) definem que essa abordagem segue, principalmente, três fases: exploratória; sistematização de coleta de dados e delimitação do estudo, e análise e interpretação dos resultados.

Considerando a fase exploratória realizada através dos levantamentos bibliográficos, conforme apresentado no Capítulo 2, a sistematização de coleta

seguirá com uma abordagem voltada para uma mescla de coletas qualitativas e quantitativas, a fim de verificar a experiência do usuário durante a utilização do produto, através da usabilidade, engajamento com o software e utilidade percebida.

No contexto deste trabalho, o estudo de caso focará na aplicação e utilização do sistema em um ambiente acadêmico, embora o software não esteja restrito exclusivamente a esse campo. Esse enfoque permitirá uma avaliação aprofundada de como o software atende às necessidades dos pesquisadores, voltados principalmente para as suas experiências ao utilizar o produto e suas funcionalidades.

A pesquisa foi conduzida por meio da utilização do software por um grupo seleto de usuários durante um período específico. Após esse período, os usuários foram convidados a responder um questionário com perguntas objetivas, visando a coleta de métricas quantitativas (Descritos no capítulo 3.3.2). Em seguida, houve algumas questões dissertativas, destinadas à coleta de dados mais subjetivos, permitindo uma compreensão mais profunda das suas percepções e experiências com o software.

Com esses indicadores, é possível delimitar o escopo do estudo para avaliar o grau de maturidade do produto. Dessa forma, os resultados analisados focaram em responder algumas perguntas principais, como: “O software atende de forma eficaz às necessidades de seus usuários?”; “Quais aspectos da interface e das funcionalidades precisam ser aprimorados?”; e “Como os usuários percebem a curva de aprendizado e a facilidade de uso do software?”.

Essas questões guiarão a análise final, oferecendo uma visão clara sobre os pontos fortes do produto e os aspectos que requerem melhorias. Além disso, ao cruzar os dados qualitativos e quantitativos, será possível identificar padrões de uso, dificuldades recorrentes e sugestões que podem servir como base para futuras atualizações do software, garantindo que ele evolua de acordo com as demandas reais dos pesquisadores. Esse estudo de caso, portanto, não apenas avalia a versão atual do produto, mas também orienta seu desenvolvimento contínuo.

3.3.2 Métricas de sucesso

Dessa forma, para garantir uma avaliação abrangente, os métodos de avaliação cobrem aspectos qualitativos e quantitativos do software. Esses métodos incluem:

a) Conformidade com os requisitos:

A conformidade com os requisitos tem o objetivo de verificar se o software atende a todas as especificações funcionais e não funcionais definidas durante a fase de planejamento e análise de requisitos (Sommerville, 2019). Este processo envolve a validação de cada funcionalidade implementada em comparação com os requisitos documentados, utilizando técnicas de verificação como revisões de código, testes automatizados e manuais. A conformidade garante que o software cumpre os objetivos para os quais foi projetado, evitando a presença de funcionalidades ausentes ou implementações incorretas (Marques; Silva; Bernardino, 2024);

b) Satisfação dos usuários:

A satisfação dos usuários mede a percepção e a aprovação do software por parte dos usuários finais (Albert; Tullis, 2022). Essa métrica é avaliada por meio de questionários e entrevistas que coletam feedback sobre a experiência geral com o software, a facilidade de uso, a eficácia das funcionalidades e a utilidade do sistema na condução das atividades de codificação e análise qualitativa (Tullis; Albert, 2013). A satisfação dos usuários é um indicador importante do sucesso do software, pois reflete diretamente a aceitação e o valor percebido pelos pesquisadores que o utilizam.

3.3.3 Coleta dos dados

A coleta de dados, portanto, seguirá uma forma estruturada de execução, principalmente para as avaliações com os usuários. Desse modo, a condução da coleta dos dados é dada pelos seguintes passos:

a) Apresentação da proposta e fornecimento de recursos;

Antes do início da coleta de dados, é essencial que os participantes estejam plenamente informados sobre o objetivo da avaliação e sobre os recursos que serão fornecidos para a execução dos testes. Este passo envolve uma apresentação detalhada da proposta do software, destacando suas funcionalidades principais, o propósito dos testes e a importância da participação dos usuários na melhoria do sistema. Além disso, é oferecido os recursos necessários para a utilização do software, incluindo um exemplo para ser codificado com o auxílio do software.

b) Avaliação de usabilidade com *System Usability Scale* (SUS);

O SUS é um instrumento utilizado para avaliar a usabilidade de sistemas de software (Brooke, 1995). Ele consiste em um questionário de 10 itens que os participantes respondem após utilizarem o software. Cada item é avaliado em uma escala de 1 a 5, baseados na escala postulada por Likert (1932), onde 1 significa "Discordo Fortemente" e 5 significa "Concordo Fortemente". A SUS fornece uma pontuação geral que pode ser interpretada para avaliar a percepção de usabilidade pelos usuários em três principais pilares: efetividade; eficiência e satisfação.

O questionário SUS - Anexo A, adaptado de Brooke (1995) - fornece algumas informações em que é possível relacionar as questões com algumas heurísticas de Nielsen (1994), como a Facilidade de aprendizagem: 3, 4, 7 e 10; Eficiência: 5, 6 e 8; Facilidade de memorização: 2; Minimização dos erros: 6 e Satisfação: 1, 4, 9; permitindo assim, uma avaliação estruturada dos resultados do produto.

c) Avaliação de satisfação com *Net Promoter Score* (NPS);

Em conjunto com o SUS, também foi utilizada a métrica NPS, para medir a lealdade e a satisfação dos usuários com o software. O NPS foi baseado em uma única pergunta: “Em uma escala de 0 a 10, qual a probabilidade de você recomendar este software a um amigo ou colega?" (Reichheld; Markey, 2011). Os usuários foram categorizados como promotores (9 e 10), neutros (7 e 8) e detratores (1 a 6), oferecendo um indicador claro do nível de aceitação entre os usuários, como mostra a Figura 6.

Figura 6 - Escala NPS



Fonte: Adaptado de (Silva, 2020)

A partir das notas dadas pelos clientes é calculada a sua nota de NPS, avaliando a taxa de satisfação dos usuários. O cálculo é feito da seguinte forma: porcentagem de clientes satisfeitos menos a porcentagem clientes insatisfeitos. O

resultado desta conta dará um número entre -100 a 100. Assim, é possível determinar o nível de lealdade e satisfação dos usuários, sendo excelente – entre 75 e 100, muito bom – entre 50 e 74, razoável – entre 0 e 49 e ruim – entre -100 e -1.

Com a aplicação desses processos e métodos, o software estará devidamente estruturado, pronto para ser desenvolvido e distribuído ao público, atendendo aos critérios de qualidade e requisitos estabelecidos. A fundamentação teórica e as escolhas metodológicas adotadas oferecem uma base sólida para a implementação de um sistema robusto e alinhado às necessidades dos usuários.

4 DESENVOLVIMENTO

O desenvolvimento do software, portanto, foi estruturado em fases, seguindo uma adaptação do SDLC tradicional ao contexto deste projeto. Assim, este capítulo aborda desde a fase de análise até a implementação, detalhando o processo de concepção e desenvolvimento do produto.

4.1 FASE DE PLANEJAMENTO E ANÁLISE

A primeira fase do projeto consistiu em uma fusão entre as etapas de planejamento e análise, na qual foram verificadas tanto a viabilidade do desenvolvimento quanto as oportunidades de mercado. Além disso, foram estabelecidas as principais definições e diretrizes do sistema.

4.1.1 Análise de viabilidade técnica

O primeiro passo para avaliar a viabilidade do sistema foi realizar uma análise de mercado, com o objetivo de identificar lacunas nos contextos já existentes. Para isso, foi conduzido um *benchmarking* de CAQDAS disponíveis no mercado, permitindo uma compreensão mais clara das oportunidades e limitações das soluções atuais.

4.1.1.1 Análise de mercado

Existem diversas ferramentas no mercado que são utilizadas para a análise de dados qualitativos, tais como ATLAS.ti (Atlas.ti, 2024), Delve (Delve, 2024), NVivo e MAXQDA (Nodari *et al.*, 2014). Essas ferramentas são reconhecidas por suas capacidades robustas de análise, mas também apresentam um valor de custo agregado que podem acabar comprometendo a sua democratização. O Quadro 1 apresenta um comparativo entre os principais softwares do segmento.

ATLAS.ti e NVivo são os mais comumente utilizados em pesquisa qualitativa (Brasil; Nascimento, 2020; Olapane, 2021) que oferecem uma variedade de ferramentas para codificação, análise e reporte. No entanto, o custo de licenças desses programas pode ser proibitivo para pesquisadores individuais e instituições com orçamentos limitados, com planos a partir de R$ 162,05 mensais.

Por outro lado, MAXQDA e Delve oferecem funcionalidades semelhantes com uma interface um pouco mais acessível, com planos a partir de 270 € anuais

(aproximadamente R$ 123 mensais), mas ainda assim representa um investimento significativo para muitos usuários potenciais (Corti; Gregory, 2011).

Quadro 1 - Comparativo de CAQDAS *closed-source*

Característica	**MAXQDA**	**ATLAS.ti**	**NVivo**	**Delve**
Suporte de Plataforma	Windows, macOS	Windows, macOS	Windows, macOS	Web-based
Importação de Dados	Textos, multimídia	Textos, multimídia	Textos, multimídia	Textos, vídeos
Análise de Texto	Codificação, anotação	Codificação, anotação	Codificação, anotação	Codificação, anotação
Análise Multimídia	Sim	Sim	Sim	Não
Visualizações	Diversas	Diversas	Diversas	Limitado
Colaboração	Avançada	Avançada	Avançada	Básica
Suporte Linguístico	Múltiplos	Múltiplos	Múltiplos	Inglês
Integração com Outras Ferramentas	SPSS, R	Limitada	SPSS, R	Nenhuma
Preço	Assinatura/ Compra	Assinatura/ Compra	Assinatura/ Compra	Assinatura

Fonte: Elaborado pelos autores.

Além dessas opções pagas, existem alternativas de código aberto, como o QualCoder e o Label Studio (com as funcionalidades apresentadas no Quadro 2). O QualCoder é um projeto que busca oferecer uma solução sem custo para a análise qualitativa, porém, a sua construção ainda carece de uma organização e documentação mais aprofundada. O Label Studio, por sua vez, é voltado para a rotulação de dados para aprendizado de máquina, e embora ofereça algumas funcionalidades aplicáveis à pesquisa qualitativa, não é completamente adequado para todos os aspectos desse tipo de análise.

Após identificar essas lacunas, o software desenvolvido foi projetado para superar algumas dessas limitações, oferecendo uma ferramenta *shareware* que facilita a codificação e a análise de dados qualitativos. A licença *shareware* permitirá

que o software seja disponibilizado gratuitamente, incentivando a contribuição da comunidade para seu aprimoramento. No entanto, alguns recursos avançados estarão disponíveis mediante assinatura, garantindo a sustentabilidade financeira necessária para a manutenção e desenvolvimento contínuo dessas funcionalidades adicionais.

Quadro 2 - Comparativo de CAQDAS *open-source*

Característica	**QualCoder**	**Label Studio**
Suporte de Plataforma	Windows, Linux	Web-based
Importação de Dados	Textos, multimídia	Textos, multimídia
Análise de Texto	Codificação, anotação	Codificação, anotação
Análise Multimídia	Limitado	Sim
Visualizações	Limitado	Limitado
Colaboração	Básica	Básica
Suporte Linguístico	Inglês	Múltiplos
Integração com Outras Ferramentas	Nenhuma	Python SDK
Preço	Gratuito	Gratuito/Pago

Fonte: Elaborado pelos autores.

4.1.1.2 Análise de produtos

Ainda com foco em uma visão de negócios, outra análise relevante a ser realizada é a de produto. De acordo com Sinek (2018), o desenvolvimento de um produto de sucesso deve começar pelo questionamento dos "porquês", ou seja, é preciso entender as razões fundamentais que motivam o projeto. Esse tipo de questionamento não apenas valida as decisões estratégicas, mas também orienta o desenvolvimento e a gestão do produto, assegurando que ele tenha um propósito claro e atenda às necessidades dos usuários de forma eficaz.

Para esse questionamento, e obter uma visão clara do produto a ser desenvolvido, foi realizado o exercício de preenchimento do *Product Vision Board* (Pichler, 2022). Essa ferramenta engloba aspectos como o público-alvo, necessidades, proposta de valor, características do produto e os objetivos de negócio. O resultado desse exercício está documentado no Apêndice A.

Assim, a visão do produto, sendo o porquê geral do projeto, é facilitar a análise

de dados qualitativos com uma ferramenta simples e acessível, que transforma dados em insights valiosos para pesquisadores e organizações. O foco está em simplificar o processo de codificação, tornando-o acessível tanto para iniciantes quanto para profissionais experientes. O público-alvo inclui pesquisadores, consultores de mercado e outros profissionais que lidam com dados qualitativos como entrevistas e transcrições.

A ferramenta busca resolver o problema da codificação e análise desses dados, oferecendo simplicidade e aprendizado rápido. Ela se destaca por trazer uma alternativa de baixo custo. O objetivo de negócio é capturar o mercado acadêmico e de pesquisa, além de expandir para consultorias e outras organizações, garantindo crescimento sustentável e retenção de clientes, com um produto acessível e fácil de usar.

Aprofundando esses conceitos, torna-se possível avaliar a viabilidade do projeto e os esforços necessários para sua implementação. Essas análises fornecerão uma base sólida para, em etapas futuras de “produtização”, apoiar o desenvolvimento de um modelo de negócios adequado.

4.1.1.3 Viabilidade e esforços de desenvolvimento

Com base na avaliação de mercado e produto, o passo seguinte foi realizar uma análise dos riscos e custos envolvidos, além de concluir sobre a viabilidade técnica para o desenvolvimento da solução.

A viabilidade financeira do projeto foi inicialmente planejada com base no desenvolvimento colaborativo, sendo mantida pela própria comunidade. No entanto, alguns recursos futuros podem gerar custos adicionais, como o uso de tecnologias externas para a sugestão automática de rotulação. Nesse sentido, o modelo de distribuição mais adequado para suportar esses custos é o *shareware*.

Em termos técnicos, a escolha de Python como linguagem de programação foi estratégica, devido à sua ampla utilização e à variedade de bibliotecas que facilitaram o desenvolvimento. A familiaridade dos programadores com a linguagem contribuiu para reduzir a curva de aprendizado, acelerando o progresso do projeto.

Quanto à avaliação dos riscos envolvidos, ao longo do planejamento e pesquisa foi possível identificar alguns pontos associados ao projeto, tais como:

O risco de complexidade no desenvolvimento e/ou incompatibilidade entre sistemas. Esses riscos técnicos são inerentes ao desenvolvimento de software, e, no nosso caso, a mitigação foi abordada por meio da definição cuidadosa dos casos de uso, requisitos e funcionalidades iniciais, assegurando que as bases do sistema fossem compatíveis com o contexto de desenvolvimento.

Outro ponto relevante é o risco de adoção, decorrente da concorrência com ferramentas já consolidadas no mercado, o que pode dificultar a migração dos usuários entre as plataformas. Para mitigar esse risco, a estratégia será investir em diferenciação, especialmente disponibilizando o software de forma gratuita, com documentação detalhada.

Um terceiro risco identificado é a falta de contribuições da comunidade. Como o projeto é de código aberto, espera-se colaboração da comunidade de desenvolvedores. Contudo, há o risco de baixa adesão ou de que as contribuições não atendam aos padrões de qualidade esperados. Para mitigar último fator, foram implementadas regras claras para a comunidade, incluindo *guidelines* de contribuição e revisão de código, além de iniciativas de integração e desenvolvimento contínuo, utilizando as ferramentas disponíveis na plataforma GitHub, como a abertura de *Issues* e a utilização do “GitHub Actions”.

Dessa forma, apesar dos riscos identificados, o software apresenta mecanismos para mitigar esses obstáculos, assegurando sua relevância e competitividade a longo prazo. O projeto, portanto, demonstra viabilidade tanto do ponto de vista técnico quanto de mercado, com potencial para atender às demandas dos usuários.

4.1.2 Definição do contexto do sistema

Com os pontos de viabilidade levantados, o passo seguinte foi definir o contexto geral do sistema. Para isso, foi necessário identificar as partes interessadas, ou *stakeholders*, diretamente relacionadas ao projeto. Esses *stakeholders* seriam a principal fonte de coleta de informações e requisitos.

Considerando a ampla gama de cenários em que o sistema pode ser utilizado, decidiu-se limitar o escopo das partes em dois grupos principais: desenvolvedores e pesquisadores focados em análises qualitativas de dados. Essa delimitação permitiu um foco mais preciso nas necessidades e expectativas desses públicos, facilitando a

definição das funcionalidades e diretrizes do sistema.

Dessa forma, foram identificados quatro *stakeholders* iniciais, descritos no Apêndice B. Por meio de entrevistas, sessões de *brainstorm* e outras formas de interação, esses indivíduos contribuíram com os principais pontos, expectativas, casos de uso e requisitos, que foram essenciais para delimitar a construção inicial do projeto. Essas contribuições permitiram um direcionamento mais claro e alinhado às necessidades reais dos usuários.

4.1.3 Definição dos requisitos

A partir dessas interações entre as partes, foi possível identificar um total de seis requisitos iniciais para o projeto, divididos entre requisitos funcionais e não funcionais. Os requisitos funcionais estão diretamente relacionados às funcionalidades oferecidas pelo sistema e aos seus componentes, portanto, eles serviram como as principais premissas para o desenvolvimento do software. São eles:

- **RF001:** Importação de Dados – permite que os usuários tragam dados em formato de texto para dentro do software;
- **RF002:** Codificação de Dados – o usuário pode associar códigos aos trechos dos textos importados;
- **RF003:** Análise de Frequência de Códigos – possibilita a visualização da utilização dos códigos e suas aparições nos textos;
- **RF004:** Gerar Relatórios – apresentar os dados de forma que possam trazer *insights* dos dados ao utilizador;
- **RF005:** Salvar e Restaurar o Projeto – garante a perpetuação do estado atual e a continuidade em outro momento.

Com esses requisitos, torna-se possível visualizar claramente as funcionalidades essenciais que o software deve oferecer para atender minimamente às expectativas dos principais usuários. Além dos requisitos funcionais, foi identificado também o requisito não funcional **RNF001** - Manutenibilidade e Evolutividade, que está diretamente relacionado à necessidade de o sistema ser desenvolvido de forma a facilitar sua manutenção contínua e permitir futuras expansões e melhorias.

Todos esses requisitos estão detalhadamente descritos no Apêndice C.

4.2 FASE DE *DESIGN*

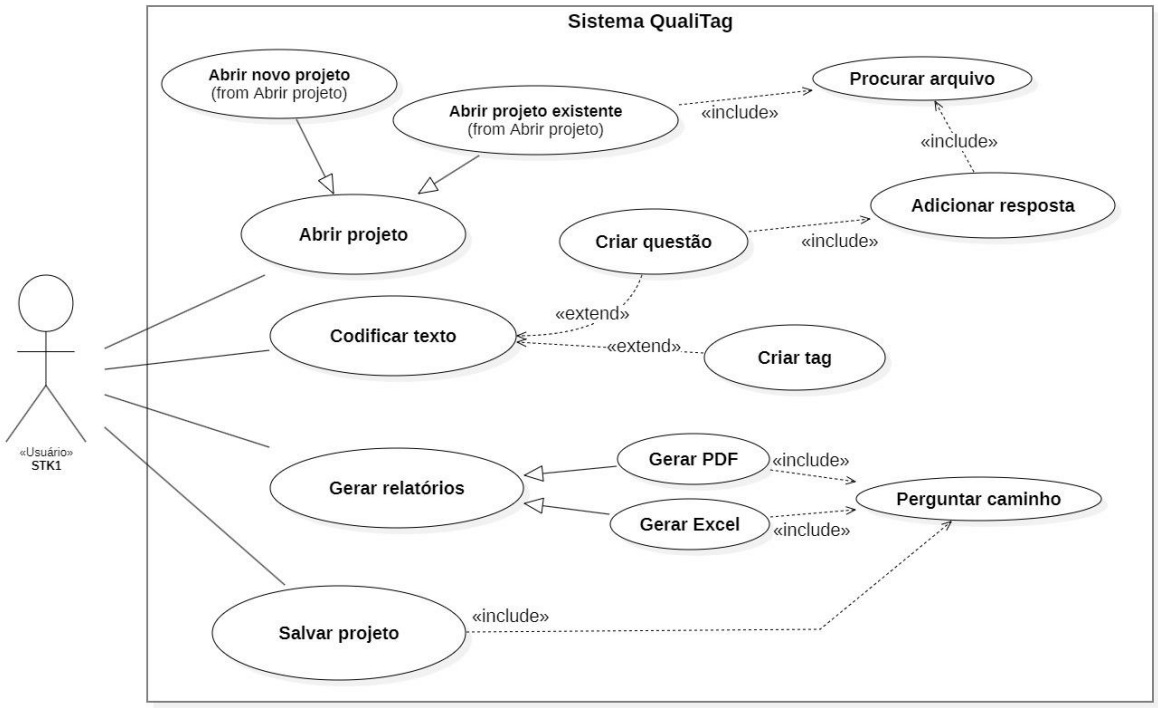
Com as funcionalidades primárias do sistema devidamente estabelecidas, foi possível idealizar e estruturar a arquitetura do software, além de definir as condições de operação e especificar os principais componentes que ele deverá incorporar.

4.2.1 Definição dos casos de uso

Logo, para dar seguimento ao processo, foram definidos os casos de uso, que descrevem de maneira clara e objetiva como os usuários interagem com o sistema para alcançar seus objetivos. Inicialmente, os casos de uso foram especificados tendo o stakeholder 1 como usuário principal, embora essa seja uma generalização aplicável a qualquer usuário, uma vez que todos terão acesso às mesmas funcionalidades.

Esses casos de uso permitem mapear as funcionalidades e comportamentos do sistema a partir da perspectiva do usuário, facilitando a compreensão das interações necessárias e garantindo que todos os requisitos identificados sejam atendidos. Os casos de uso iniciais do sistema podem ser vistos conforme a Figura 7.

Figura 7 - Casos de uso



Fonte: Elaborado pelos autores.

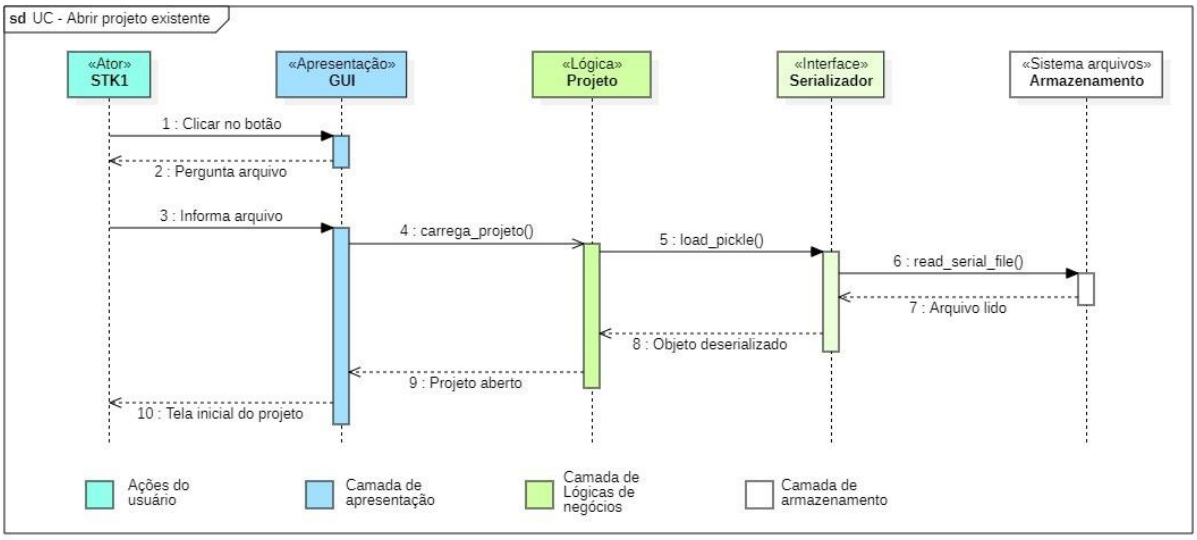
Uma descrição completa de cada caso de uso pode ser observada no Apêndice D. No presente contexto, foram definidos quatro casos de uso principais com os quais o usuário pode interagir:

- **Abrir projeto**: O usuário pode iniciar um novo projeto ou abrir um projeto já existente. Esta funcionalidade abrange tanto a criação de um novo ambiente de trabalho quanto a busca por arquivos de projetos salvos anteriormente, garantindo flexibilidade para continuar o trabalho de onde foi interrompido, atendendo ao requisito RF005;
- **Codificar um texto**: Permite ao usuário realizar a codificação de trechos de texto, associando *tags* específicas a partes relevantes do conteúdo. Essa função é essencial para organizar a informação de forma estruturada, facilitando a categorização e subsequente análise qualitativa dos dados. Além disso, essa funcionalidade pode ser expandida para incluir a criação de novas *tags* durante o processo de codificação, bem como a adição de novas perguntas repostas a partir dos dados, contemplando os requisitos RF001 e RF002;
- **Gerar relatórios**: O sistema oferece a capacidade de gerar relatórios com base nas análises realizadas. Estes relatórios podem ser exportados em formatos como PDF e Excel, garantindo flexibilidade na apresentação e compartilhamento dos resultados. Esse caso de uso contempla os requisitos RF003 e RF004;
- **Salvar projeto:** Por fim, o usuário pode salvar o projeto a qualquer momento, garantindo que todas as alterações feitas, como a codificação de textos e geração de relatórios, sejam armazenadas em disco. Isso permite que o projeto seja retomado posteriormente sem perda de informações, garantindo parte do requisito RF005.

Ainda em relação aos casos de uso, nota-se as suas relações sequenciais. Essas relações, de modo implícito, transportam a informação entre as camadas do sistema. Para exemplificar, a Figura 8 traz um diagrama sequencial para o caso de uso onde o usuário está realizando a abertura de um projeto já existente.

Nesse caso, a informação é transportada entre as três camadas do sistema: apresentação, lógica de negócios e armazenamento. Na versão inicial do projeto, o sistema opera inteiramente de forma local, ou seja, a camada de armazenamento é implementada no próprio sistema de arquivos do sistema operacional do usuário.

Figura 8 - Diagrama sequencial para abrir projeto existente



Fonte: Elaborado pelos autores.

Essa arquitetura permite que o desenvolvimento do sistema seja realizado em etapas independentes, possibilitando, por exemplo, o desenvolvimento paralelo das camadas de apresentação e lógica de negócios, otimizando o processo de implementação.

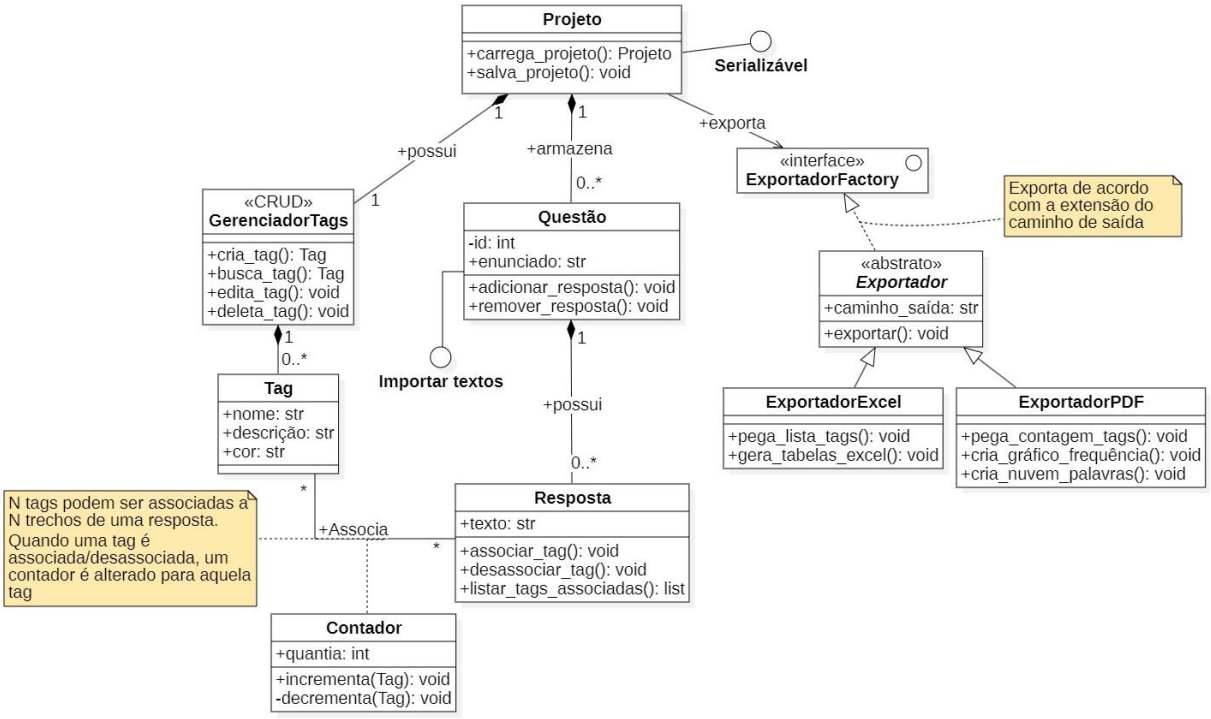
4.2.2 Especificação dos componentes

Após a definição dos casos de uso, foi possível abstrair os principais componentes do sistema, que constituirão, sobretudo, a camada de lógica de negócios. Esses componentes são responsáveis por dar vida ao software, funcionando como o núcleo que processa as regras e operações do sistema.

Para garantir uma transição eficiente entre a idealização e a implementação, foram utilizados modelos em diagramas de classe, que descrevem de forma clara como a implementação deverá ocorrer. O diagrama de classes principal, relacionado à camada de lógica de negócios do sistema, é expresso conforme a Figura 9.

Conforme mostra o diagrama, o sistema será centralizado na classe “Projeto”. Essa classe por si só tem como objetivo gerenciar os recursos para a serialização e perpetuação das análises qualitativas, para atender especificamente ao requisito RF005 . Ela funcionará como o repositório central, agregando as informações necessárias para a organização e gerenciamento do projeto. A partir dela, haverá uma relação de composição ou associação com as demais classes e interfaces e suas respectivas funções.

Figura 9 - Diagrama de classes principal



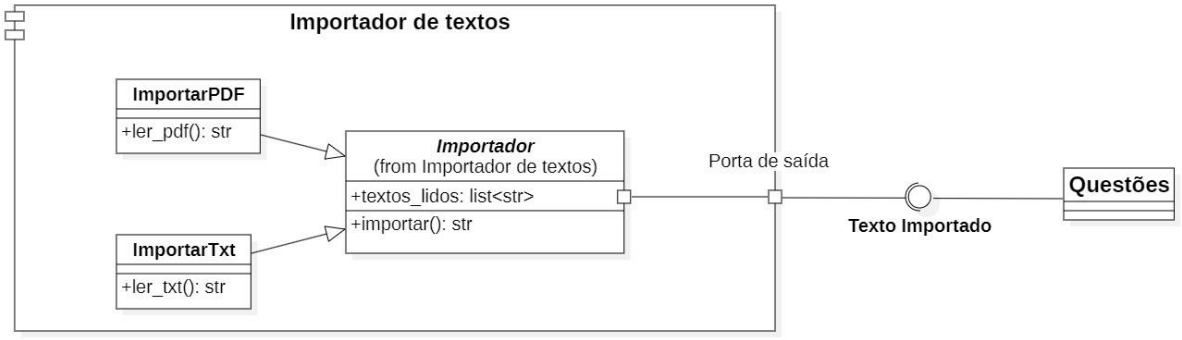
Fonte: Elaborado pelos autores.

Ainda como o apresentado, a classe principal é composta por zero ou mais questões, que, por sua vez, são compostas por zero ou mais respostas, bem como o projeto também é composto por um gerenciador de *Tags*, que as cria, procura, edita ou deleta do sistema. Essas abstrações de acordo com cada responsabilidade, seguem os princípios de responsabilidade única da metodologia SOLID para a escrita de códigos (Moeller, 2024), onde cada classe deve ter apenas um objetivo primário.

Além disso, para que questões e respostas sejam geradas a partir de documentos fornecidos pelos usuários, tornou-se necessário criar interfaces de importação especializadas. Essas interfaces foram projetadas para lidar com diferentes tipos de documentos, como arquivos de texto (“.txt”) e PDFs.

Cada classe de importação, portanto, foi estruturada para se especializar no formato de documento correspondente, garantindo que os dados sejam corretamente extraídos e integrados ao sistema, de acordo com o formato de origem, conforme mostra o diagrama de componentes da Figura 10. Nele, é possível ver, além da integração entre as interfaces, que a classe abstrata “Importador” pode ser especializada em subclasses específicas, porém a exportação depende apenas da classe abstrata. Essa condição garante outro princípio do SOLID, a inversão de dependências (Aragão, 2019).

Figura 10 - Diagrama de componente para importador de textos



Fonte: Elaborado pelos autores.

Um esquema semelhante é observado na exportação do projeto, conforme o diagrama de classes principal, onde a dependência se baseia em uma classe abstrata, que se especializa conforme a necessidade. Nessa estrutura, o sistema utiliza o conceito de "*Factory*" dentro do padrão de *design* (Shvets, 2021), facilitando a criação de objetos sem a necessidade de especificar a classe exata que será instanciada. As especializações implementam os mesmos métodos da classe pai, adaptados ao seu contexto específico. Essa abordagem polimórfica proporciona maior flexibilidade e extensibilidade, permitindo que a exportação do projeto seja facilmente adaptada a diferentes formatos, como PDF ou Excel, de maneira modular maneira modular.

O sistema também necessitou de um componente fundamental: a abstração dos códigos, ou *tags*, que serão associadas aos dados ao longo da jornada do usuário. Essas devem ser gerenciadas por uma classe específica, em conformidade com o princípio de responsabilidade única. As *tags* ainda podem ser associadas à trechos específicos nas respostas, essa associação ainda possui uma classe controladora para monitorar a quantidade de vezes que aquela etiqueta foi associada no projeto.

4.2.3 Interface gráfica do usuário

A última etapa do *design* para a construção da versão primária do software, é a criação de uma interface gráfica, com a qual o usuário interagirá diretamente com o sistema, representando a própria camada de apresentação. Como o foco principal deste trabalho está nas funcionalidades do software, e não na interface em si, não foi realizado um estudo aprofundado sobre experiência do usuário ou usabilidade.

Dessa forma, a idealização da interface buscou ser simples, priorizando a clareza sobre a sofisticação visual. Para isso, evitou-se o uso de ícones ou imagens, optando por um layout mais direto, com textos descrevendo cada funcionalidade. A

organização da interface foi baseada em *wireframes*, que serviram como guias para dispor os elementos de forma que atendam ao propósito do MVP.

O software, portanto, possuirá três telas principais com as quais o usuário irá interagir. A primeira, sendo a tela inicial de boas-vindas, apresentará duas opções, onde o usuário poderá escolher entre começar um projeto do zero, ou continuar um projeto salvo previamente, conforme mostra o *wireframe* da Figura 11.

Figura 11 - Tela inicial



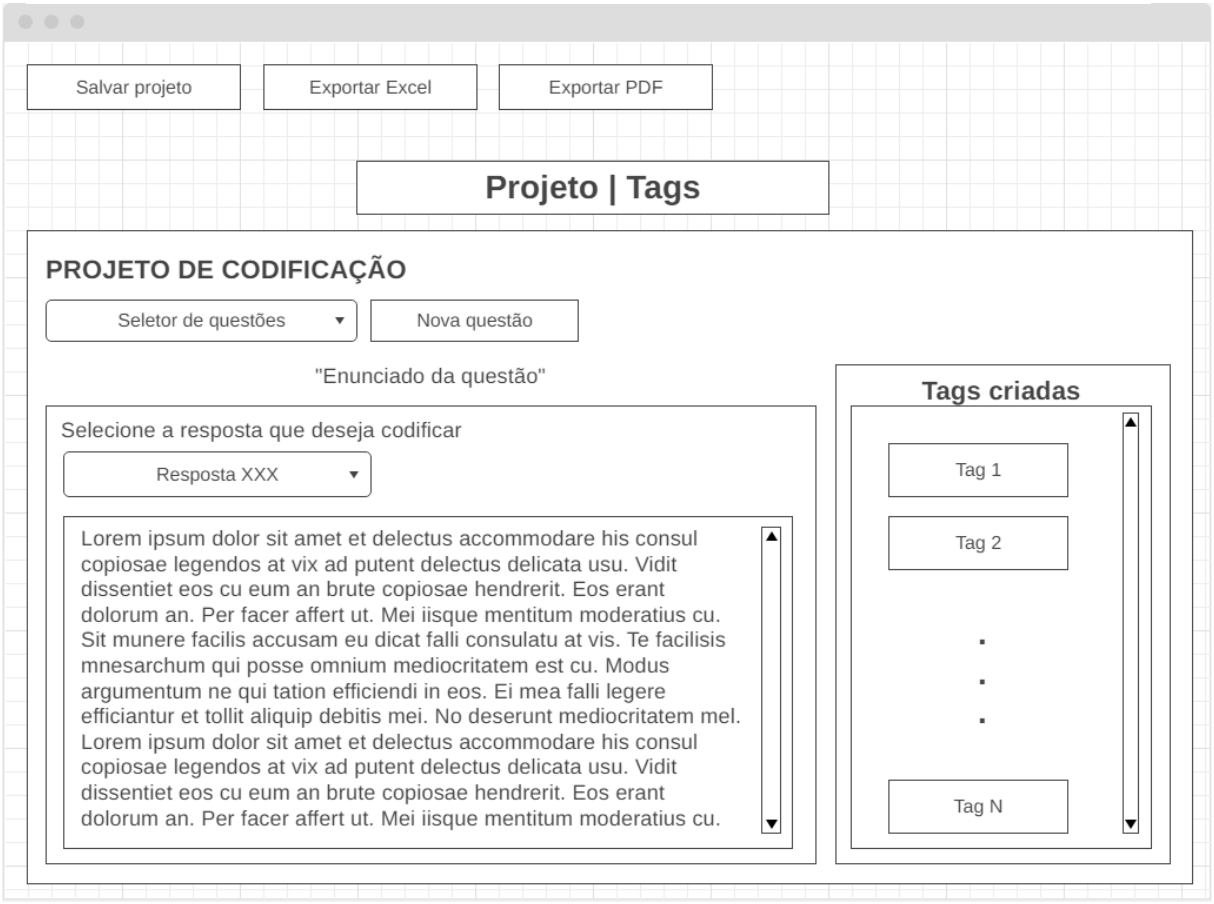
Fonte: Elaborado pelos autores.

A segunda tela será onde o usuário realizará o processo de codificação propriamente dito. Assim, ela contará com um menu superior, onde estarão disponíveis as opções de salvamento e exportação, garantindo fácil acesso a essas funcionalidades durante o uso. Logo abaixo, haverá um menu de navegação entre as diferentes abas do software, permitindo ao usuário alternar rapidamente entre as funções disponíveis.

No conteúdo desta aba inicial de codificação, o usuário encontrará um seletor de questões, que permite escolher a questão em análise, além de um botão para criar questões. À esquerda, haverá uma lista lateral com todas as *tags* previamente criadas, facilitando a organização e aplicação das mesmas durante o processo. Abaixo do

seletor de questões, o usuário terá acesso a um seletor de respostas, onde poderá escolher qual resposta deseja codificar, e uma caixa de texto que exibirá o conteúdo da resposta selecionada, onde o processo de codificação será executado, conforme apresentado na Figura 12.

Figura 12 - Tela de codificação



Fonte: Elaborado pelos autores.

A última aba será dedicada ao gerenciamento das *tags* criadas. Nessa tela, o usuário encontrará uma tabela que exibirá o nome de cada uma, um botão para deletar alguma específica, e a quantidade de vezes que cada uma foi associada aos dados ao longo do projeto. Além disso, um botão para criação também estará disponível, permitindo a adição de novos códigos conforme a análise avança.

Uma funcionalidade adicional nesta aba permitirá que, ao clicar sobre o nome na lista, a barra lateral exiba a descrição completa da *tag* selecionada, proporcionando mais informações e contexto sobre o seu uso e significado. Isso oferece ao usuário uma visão mais detalhada, ajudando no gerenciamento e controle dos códigos utilizados no projeto.

Essa organização facilita a manutenção do processo, permitindo que o usuário

tenha controle total sobre o processo de codificação e a flexibilidade para criar, modificar ou excluir *tags* conforme necessário. A representação dessa tela pode ser vista na Figura 13.

Figura 13 - Tela de gerenciamento de *tags*



Fonte: Elaborado pelos autores.

4.3 FASE DE IMPLEMENTAÇÃO

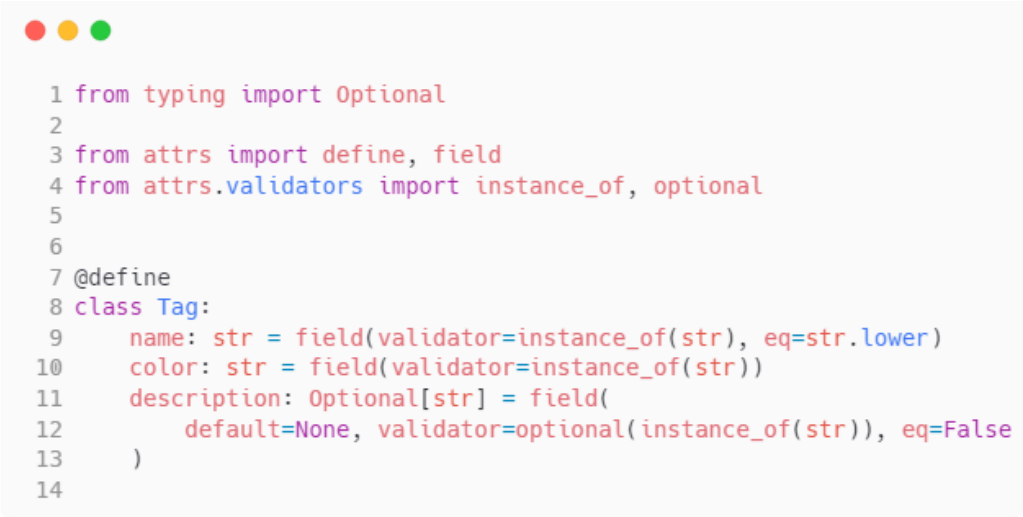
Com a fase de estruturação do projeto concluída, foi possível iniciar a implementação. Como mencionado anteriormente, a linguagem escolhida para o desenvolvimento foi Python, devido à sua gama de bibliotecas e funcionalidades que facilitaram a construção da solução. Além disso, a familiaridade dos desenvolvedores com a linguagem contribuiu para acelerar o processo de implementação.

4.3.1 Implementação dos componentes

A partir dos modelos de domínio, apresentados no capítulo 4.2, foi possível traduzi-los diretamente em código. Em casos específicos, onde a abstração das

classes apresentava apenas atributos sem lógica adicional, como as *tags*, utilizou-se a técnica de *dataclasses* do Python. Essa abordagem permitiu uma codificação rápida, facilitando a definição e tipagem dos atributos das classes de forma concisa e clara. Vide o exemplo do código da classe “*Tag*” na Figura 14.

Figura 14 - Implementalação da classe "*Tag*"



```
1 from typing import Optional
2
3 from attrs import define, field
4 from attrs.validators import instance_of, optional
5
6
7 @define
8 class Tag:
9     name: str = field(validator=instance_of(str), eq=str.lower)
10    color: str = field(validator=instance_of(str))
11    description: Optional[str] = field(
12        default=None, validator=optional(instance_of(str)), eq=False
13    )
14
```

Fonte: Elaborado pelos autores.

Nota-se que, para essa classe, o único atributo utilizado para a comparação de igualdade entre os objetos é o nome da *tag*, sendo essa comparação insensível a maiúsculas e minúsculas (como indicado pelo uso de *“eq=str.lower*” na linha 9). Essa característica foi importante para o gerenciador de *tags*, pois permitiu uma melhor detecção de possíveis erros do usuário, como a criação acidental de dois códigos com o mesmo nome, mas com variações de capitalização. Dessa forma, o sistema se torna mais robusto, evitando duplicidades indesejadas e garantindo maior consistência na organização das informações.

De maneira semelhante, nos casos em que era necessária a especialização das classes, como no módulo de importação, foi utilizada o conceito de herança em Python. Esse recurso permitiu criar subclasses especializadas que herdavam atributos e métodos comuns da classe principal, garantindo reutilização de código e uma arquitetura mais organizada, conforme mostra a Figura 15.

Além disso, para a seleção do método de importação a ser utilizado, o código adotou o padrão criacional conhecido como “*Factory Method*”. Esse padrão cria uma interface que, com base em parâmetros específicos (como o tipo de arquivo a ser importado), retorna a subclasse apropriada para realizar a tarefa. Essa abordagem

garante escalabilidade ao sistema, facilitando a adição de novos tipos de importadores sem a necessidade de modificar o código existente. Dessa forma, o princípio de abertura para extensão e fechamento para modificação é mantido, promovendo a evolução do software de forma sustentável.

Figura 15 - Importadores e factory method

```python
# Importador base
class ImporterBase(ABC):
    @abstractmethod
    def import_data(self):
        pass

# Importador PDF
class PDFImporter(ImporterBase):

    def import_data(self) -> Answer:
        text = ""
        with open(self.filepath, "rb") as f:
            reader = PdfReader(f)

            for page in reader.pages:
                text += page.extract_text()

        return Answer(text)

# Importador TXT
class TXTImporter(ImporterBase):

    def import_data(self):
        text = ""
        with open(self.filepath, "r", encoding="utf-8") as f:
            text = f.read()
        return Answer(text)

# Factory method para os métodos de importação
def import_data(filepath: str):
    _, ext = splitext(filepath)

    if ext == ".pdf":
        importer: ImporterBase = PDFImporter(filepath)
    elif ext == ".txt":
        importer: ImporterBase = TXTImporter(filepath)
    else:
        raise ValueError(f"Unsupported file format: {ext}")

    return importer.import_data()

```

Fonte: Elaborado pelos autores.

A implementação completa do projeto, incluindo todos os códigos e demais arquivos, está disponível no repositório público no GitHub[1]. O repositório contém toda

[1] GitHub, Disponível em: https://github.com/PiresFelipe/QualiTag. Acesso em: 30 de set. de 2024

a estrutura do software, documentação e instruções detalhadas sobre como instalar e utilizar o sistema, permitindo que outros desenvolvedores ou interessados possam acessar, contribuir ou adaptar o projeto conforme necessário.

4.3.2 Criação de testes unitários

Com os componentes devidamente implementados, realizou-se a criação de testes unitários para garantir a funcionalidade correta de cada parte do sistema. Para auxiliar nessa tarefa, foi utilizada a biblioteca "pytest", uma ferramenta reconhecida no ecossistema Python para a automatização de testes.

No contexto deste projeto, onde funcionalidades críticas como a codificação de dados, gerenciamento de *tags* e importação/exportação de documentos são fundamentais, os testes unitários asseguram que cada módulo opere conforme o esperado. Além disso, ao garantir que o sistema esteja adequadamente testado, minimizam-se os riscos de introdução de erros durante o desenvolvimento ou implementação de novas features.

Dessa forma, cada componente mencionado foi submetido a testes unitários, que verificavam o correto funcionamento de suas respectivas funções, assegurando que atendessem ao nível mínimo de execução esperado para o sistema. A criação dos testes foi planejada para cobrir critérios específicos, focando principalmente na cobertura de instruções e ramificações. Isso garantiu que a maioria das instruções, linhas de código e ramificações fosse devidamente verificada, assegurando uma alta confiabilidade e robustez do software.

Para garantir a cobertura necessária, os testes foram planejados de modo a percorrer todas, ou a maioria, das linhas de cada código, assegurando o correto funcionamento dos componentes. Em alguns casos, foram utilizados diagramas que ilustravam os caminhos possíveis e as condições enfrentadas pelo código, facilitando a identificação de ramificações. Como resultado, foi alcançada uma cobertura de aproximadamente 84% do código, superando o mínimo estabelecido de 80% para este software. A Tabela 1 apresenta o relatório completo da cobertura de testes.

Observa-se uma baixa cobertura no arquivo “project.py”, mas isso se deve ao fato de que as suas operações básicas são de serialização dos dados, e ela é uma classe composta pelas demais, cujo as quais tem cobertura quase que total de suas instruções.

Tabela 1 - Cobertura dos testes por arquivo

Arquivo	Instruções	Faltantes	Excluídas	Cobertura (%)
\src\project.py	61	38	0	38
\src\exporter\pdf_exporter.py	10	1	55	90
\src\tags\tags_manager.py	36	2	0	94
\src__init__.py	5	0	0	100
\src\exporter__init__.py	5	0	11	100
\src\exporter\exporter_base.py	2	0	7	100
\src\exporter\xlsx_exporter.py	2	0	22	100
\src\importer__init__.py	13	0	0	100
\src\importer\importer_base.py	1	0	4	100
\src\importer\pdf_importer.py	13	0	0	100
\src\importer\txt_importer.py	10	0	0	100
\src\questions__init__.py	3	0	0	100
\src\questions\answer.py	30	0	0	100
\src\questions\question.py	20	0	0	100
\src\tags__init__.py	4	0	0	100
\src\tags\tag.py	8	0	0	100
\src\tags\tags_counter.py	23	0	2	100
Total	**246**	**41**	**101**	**84%**

Fonte: Elaborado pelos autores.

Além disso, os testes unitários desempenharam um papel fundamental no processo de desenvolvimento e integração contínua do software. Eles foram integrados ao pipeline de integração contínua, garantindo que cada nova contribuição ao código, seja ela uma correção ou uma nova funcionalidade, passe por uma validação. Todas as futuras contribuições ao projeto deverão ser submetidas a esses testes unitários, assegurando que as novas alterações não comprometam a integridade e a funcionalidade já estabelecidas no sistema.

4.3.3 Desenvolvimento e integração contínua

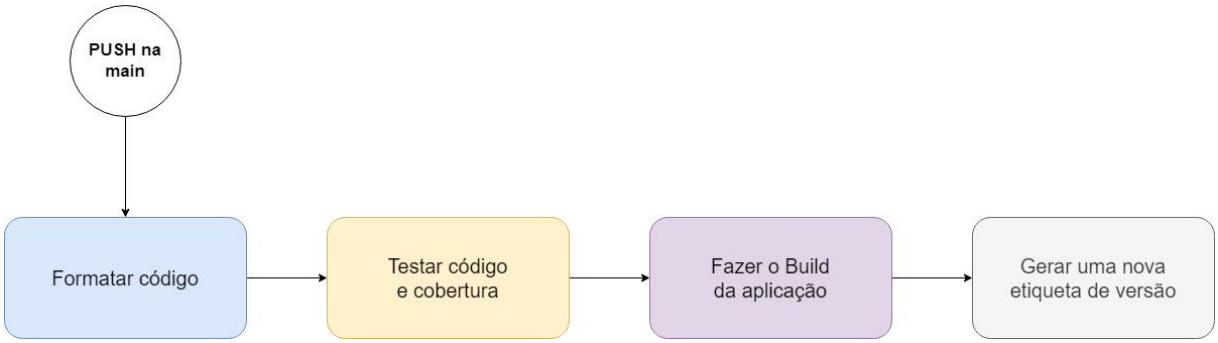
Ainda em termos de desenvolvimento e integração contínua, o presente projeto adotou diretrizes específicas para essa abordagem, o que não apenas facilita a manutenção do software, mas também estabelece parâmetros claros para a continuidade do desenvolvimento ao longo do tempo. Essas diretrizes incluem práticas como:

- **Versionamento de código**: Utilização de um sistema de controle de versão, no caso o Git, para acompanhar todas as mudanças no código, permitindo a colaboração de vários desenvolvedores e a reversão de versões anteriores, se necessário.

- **Testes automatizados**: Como mencionado anteriormente, todas as contribuições passam por uma suíte de testes unitários, garantindo que o software continue funcionando corretamente após cada alteração.
- ***Deploy* automatizado**: Configuração de pipelines de integração contínua para automatizar o processo de *build* e *deploy* do software, assegurando que cada versão seja testada e implantada de maneira rápida e segura.

Todo esse processo foi automatizado e gerenciado através do *GitHub Actions*, uma ferramenta para automação de fluxos de trabalho. Com o ela, foi possível configurar os pipelines que executam automaticamente os testes unitários, builds e deploys a cada nova contribuição ao repositório. Assim, o projeto possui uma pipeline descrita conforme a Figura 16.

Figura 16 - *Pipeline* de desenvolvimento e integração



Fonte: Elaborado pelos autores.

A imagem mostra que o pipeline ocorre em quatro estágios, sendo acionado automaticamente quando uma nova alteração é adicionada à ramificação principal do versionador de código. O primeiro estágio, formatação de código, garante que o código siga os padrões da comunidade Python, utilizando ferramentas de formatação de texto. O segundo estágio realiza a testagem e verificação da cobertura de testes, garantindo que o código seja validado de acordo com os critérios de cobertura estabelecidos. O terceiro estágio é responsável pela construção do software, gerando os arquivos executáveis, entre outros artefatos necessários. Por fim, o quarto estágio cria uma etiqueta de versão automática para a alteração realizada (ex.: v1.0, v2.0, etc.), permitindo um controle entre as versões.

4.3.4 Documentação dos componentes

Por fim, como parte dos requisitos de manutenibilidade e evolutividade do software, foi adotada a prática de utilizar *docstrings*, um padrão amplamente utilizado

pela comunidade Python. As *docstrings* são inserções de documentação diretamente no código, explicando o propósito e o funcionamento de cada componente, como classes, métodos e funções. Isso garantiu que o código seja autoexplicativo e facilite a compreensão tanto para os desenvolvedores atuais quanto para futuros colaboradores.

Em complemento a essa adoção, o projeto contou com o suporte da biblioteca MkDocs[2]. Essa biblioteca facilitou a criação de documentação, mesclando as *docstrings* do código com arquivos escritos em Markdown, permitindo a geração automática de um manual de usuário completo e estruturado.

Essa abordagem ofereceu uma documentação integrada, onde tanto as explicações técnicas dos componentes quanto as instruções de uso para os usuários são facilmente acessíveis. Ao utilizar o MkDocs, foi possível garantir que o manual evolua junto com o código, mantendo a documentação sempre atualizada à medida que o software avança.

Com essas etapas, foi possível alcançar a versão inicial do software. Esta versão representa os resultados da primeira iteração do SDLC, onde um conjunto restrito de requisitos foi selecionado e implementado. O resultado da implantação, bem como um breve tutorial de utilização da aplicação pode ser visto no Apêndice E.

Após a conclusão dessa fase, o produto passou por uma avaliação, com o objetivo de identificar pontos de melhoria e definir possíveis requisitos para a próxima iteração do desenvolvimento.

[2] MKDOCS. **MkDocs: Project documentation with Markdown**. 2024. Disponível em: https://www.mkdocs.org. Acesso em: 30 de set. de 2024.

5 RESULTADOS

A última fase dessa primeira iteração de desenvolvimento, portanto, envolveu as avaliações e testes de aceitação com o usuário. Nessa fase, o foco foi verificar se o software atendeu às expectativas dos usuários e validar seu desempenho em condições reais de uso.

Este capítulo apresentará os resultados obtidos na fase de avaliação, que foi estruturada em três etapas principais: a utilização do software pelos usuários, a coleta de dados quantitativos por meio dos questionários descritos na seção 3.3, e, por fim, a aplicação de questionários com perguntas dissertativas para a obtenção de dados qualitativos.

Para as avaliações, foram convidados 10 usuários, todos com perfil acadêmico, compostos por graduandos em Ciências da Computação ou áreas correlatas, além de mestrandos e doutores também dessa área. Houve uma distribuição entre aqueles que já haviam trabalhado com dados qualitativos e os que estavam realizando essa tarefa pela primeira vez. Aproximadamente 60% dos usuários já tinham experiência com esse tipo de dado, enquanto 40% estavam entrando em contato com a análise qualitativa pela primeira vez.

5.1 RESULTADOS QUANTITATIVOS

Os dados quantitativos, portanto, foram coletados por meio dos questionários SUS e NPS. O SUS foi utilizado para medir a usabilidade do software, fornecendo uma avaliação objetiva sobre a facilidade de uso, eficiência e satisfação dos usuários. Já o NPS foi aplicado com o objetivo de medir a lealdade e a disposição dos usuários em recomendar o software, refletindo o grau de aceitação do produto.

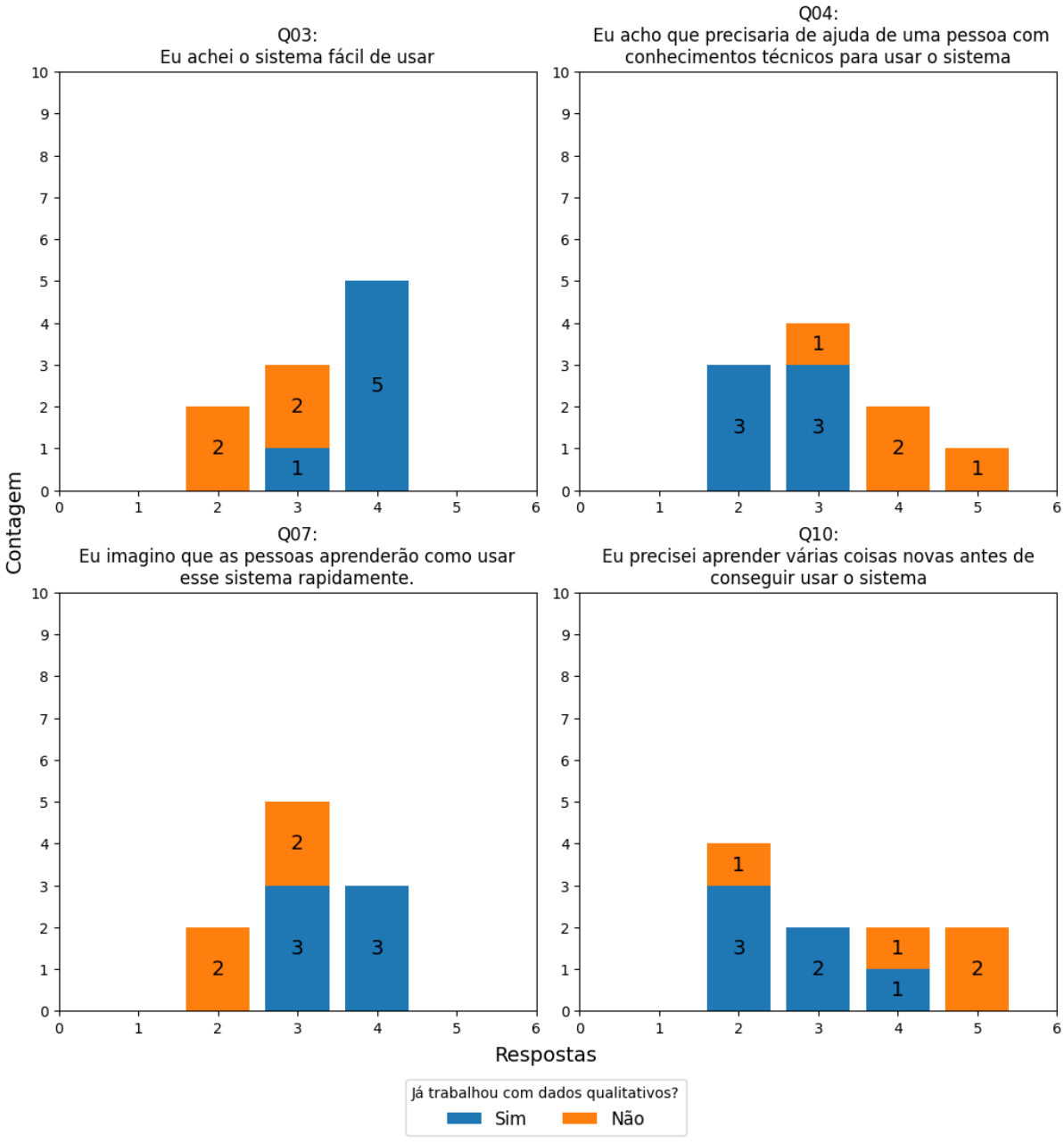
5.1.1 Resultados *System Usability Scale*

Os resultados desse questionário foram divididos relacionando-se principalmente com algumas heurísticas de Nielsen, conforme sugerido por Barros (2022).

5.1.1.1 Facilidade de aprendizagem

A primeira heurística avaliada refere-se, portanto, à facilidade de aprendizagem do usuário com o produto. Esse critério é fundamental para entender o quão rapidamente novos usuários conseguem se familiarizar com o software e realizar suas tarefas sem grandes dificuldades. Para mensurar esse aspecto, são consideradas as questões 3, 4, 7 e 10 do questionário SUS, que abordam diretamente a curva de aprendizagem do sistema. A distribuição das respostas relacionadas a essas questões podem ser observadas na Figura 17.

Figura 17 - Resultados questões 3,4, 7 e 10

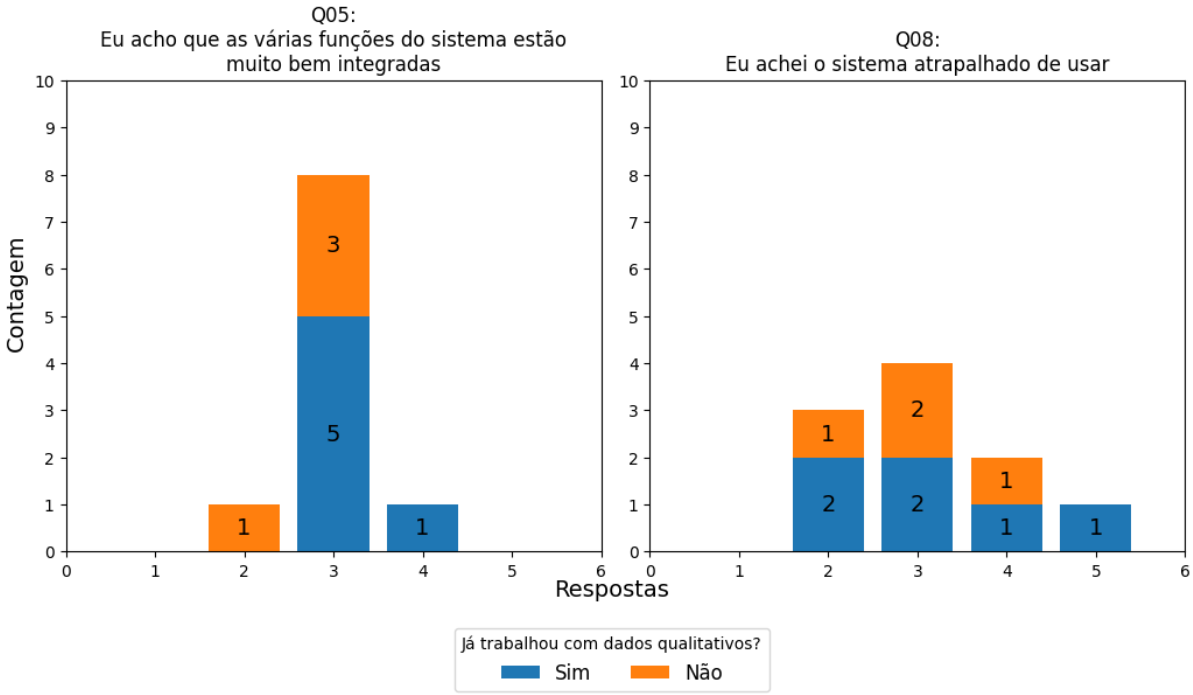


Fonte: Elaborado pelos autores.

5.1.1.2 Eficiência

A próxima heurística avaliada diz respeito à eficiência do produto, ou seja, à capacidade do software de cumprir suas funções básicas, de forma a transmitir as informações de maneira clara e adequada tanto para usuários leigos quanto para os mais experientes. As questões 5 e 8são utilizadas para medir esse aspecto. A distribuição dos resultados dessas questões pode ser visualizada na Figura 18.

Figura 18 - Resultado questões 5 e 8

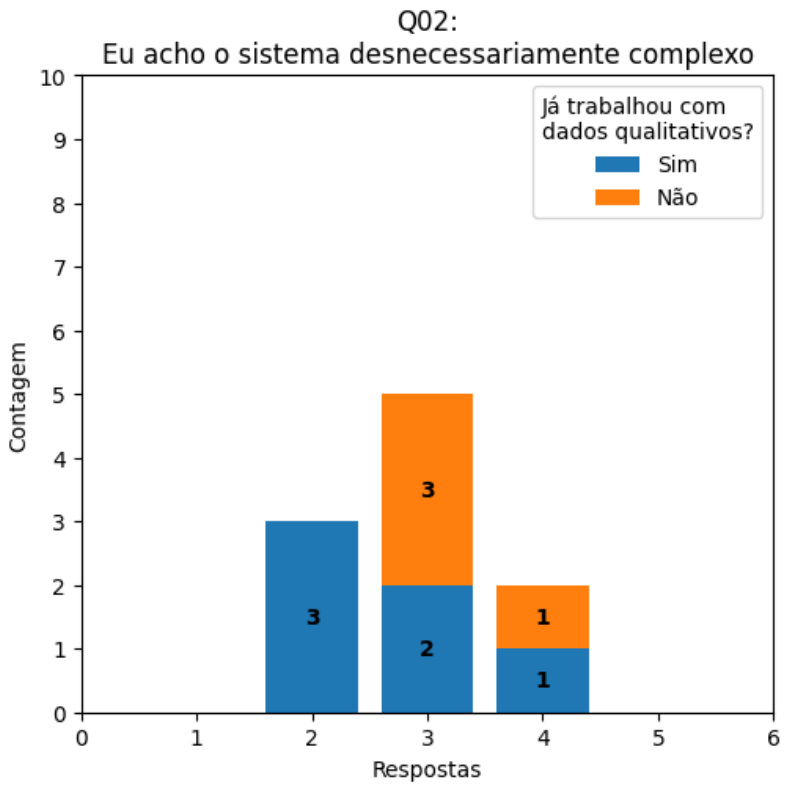


Fonte: Elaborado pelos autores.

5.1.1.3 Facilidade de memorização

Outro aspecto importante é a complexidade do sistema, que pode dificultar significativamente a capacidade do usuário de reconhecer e se lembrar dos padrões de utilização. Esse aspecto está diretamente relacionado à heurística de facilidade de memorização, que avalia o quão intuitivo o sistema é após o primeiro uso, permitindo que o usuário retome suas atividades sem precisar reaprender etapas. Para essa análise, a questão 2 do questionário SUS é utilizada como indicador-chave. Os resultados dessa avaliação estão dispostos na Figura 19.

Figura 19 - Resultados questão 2



Fonte: Elaborado pelos autores.

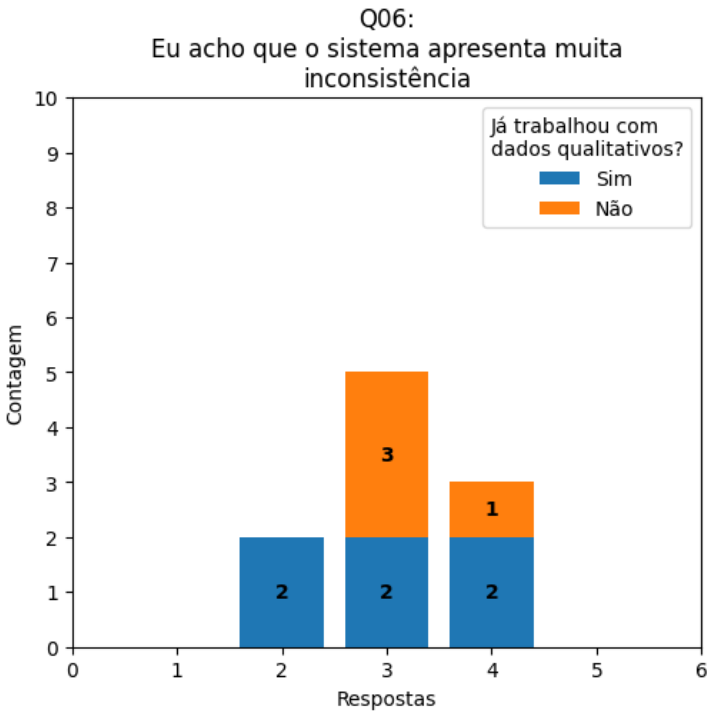
5.1.1.4 Minimização dos erros

Além desses aspectos, é fundamental mitigar os erros, sejam eles causados pelo próprio sistema ou gerados pelas ações dos usuários. A capacidade do software de lidar de maneira consistente e robusta com esses erros representa uma heurística importante, pois garante uma experiência de uso mais estável e confiável. Esse aspecto, que mede a efetividade do sistema em prevenir e corrigir falhas, é avaliado pela questão 6 do questionário. Os resultados dessa parte estão apresentados na Figura 20.

5.1.1.5 Satisfação

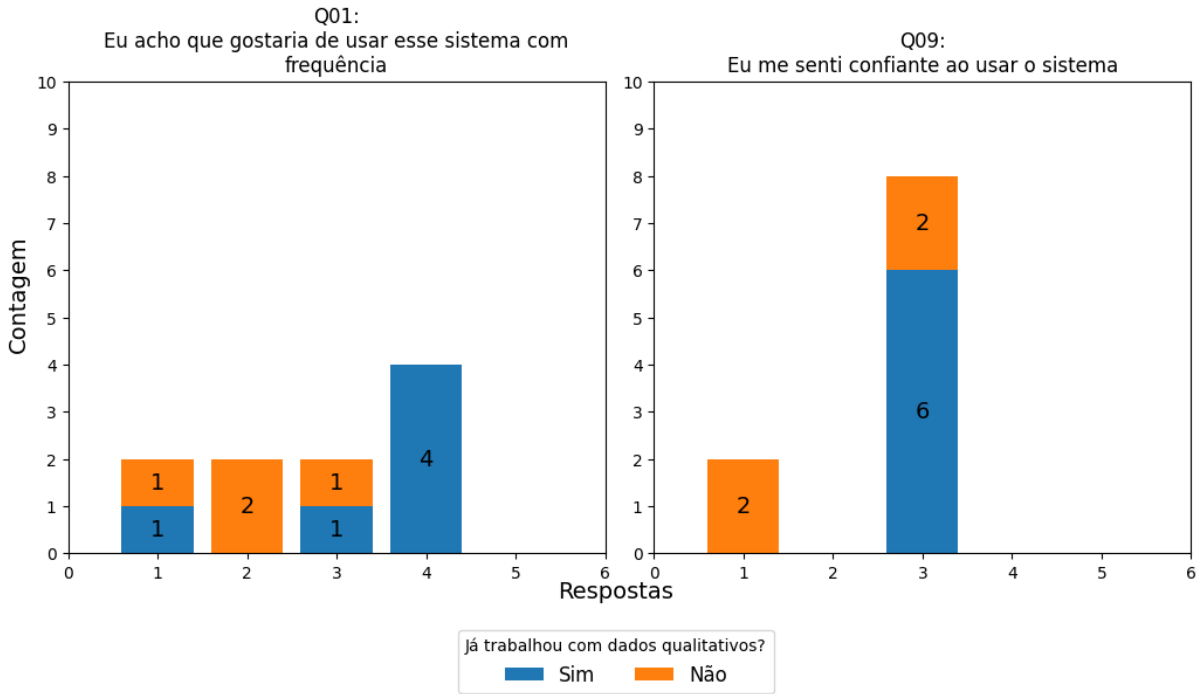
Por fim, o grau de satisfação geral do usuário com o produto, que representa sua percepção sobre a qualidade e o valor oferecido pelo software, é um indicador para avaliar o sucesso da aplicação. Esse critério reflete a experiência global do usuário, abrangendo desde a facilidade de uso até a eficiência nas funcionalidades oferecidas. O grau de satisfação geral é mensurado pelas questões 1 e 9 do questionário SUS, cujos resultados estão apresentados na Figura 21.

Figura 20 - Resultados questão 6



Fonte: Elaborado pelos autores.

Figura 21 - Resultados questões 1 e 9



Fonte: Elaborado pelos autores.

Além das heurísticas mencionadas, o SUS possui seu próprio cálculo, que avalia de forma direta a usabilidade percebida do software. Para realizar esse cálculo, os seguintes passos são seguidos:

- Para as questões ímpares (1, 3, 5, 7, 9), subtrai-se 1 da pontuação dada pelo usuário;
- Para as questões pares (2, 4, 6, 8, 10), subtrai-se a pontuação dada pelo usuário de 5;
- Soma-se os resultados de todos os itens;
- Multiplica-se o valor total por 2,5, o que resulta em uma pontuação final entre 0 e 100.

Dessa forma, o software desenvolvido atingiu uma pontuação de **51,5** no System Usability Scale para a versão inicial, indicando o nível de usabilidade percebida pelos usuários.

5.1.2 Resultados *Net Promoter Score*

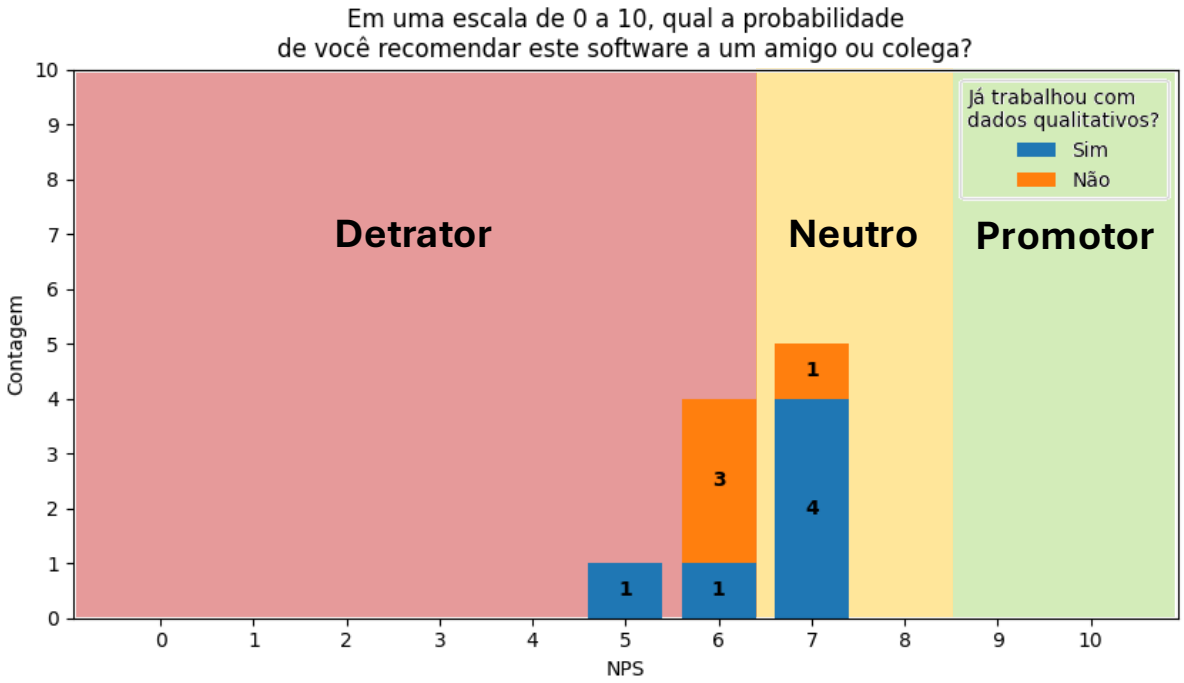
Ainda como parte dos dados quantitativos, a escala NPS foi aplicada no questionário, pedindo aos usuários que avaliassem, em uma escala de 0 a 10, o quanto acreditavam que recomendariam o software a outras pessoas. Após a coleta dos resultados, o cálculo do NPS foi realizado da seguinte forma:

Primeiro os usuários foram divididos em categorias, de acordo com a avaliação dada, sendo **detratores** (notas de 0 a 6): Usuários insatisfeitos ou que não recomendariam o software; **neutros** (notas 7 e 8): Usuários satisfeitos, mas que não necessariamente recomendariam o produto e **promotores** (notas 9 e 10): Usuários altamente satisfeitos, que provavelmente recomendariam o software. O cálculo então é realizado conforme a Equação 1.

$$NPS = \text{Promotores (\%)} - \text{Detratores (\%)} \quad (1)$$

Com base nesse cálculo, o software alcançou uma pontuação de **-50**, pois não houve nenhum promotor de fato para a versão inicial. As respostas estão dispostas conforme a Figura 22.

Figura 22 - Respostas para o NPS



Fonte: Elaborado pelos autores.

5.2 RESULTADOS QUALITATIVOS

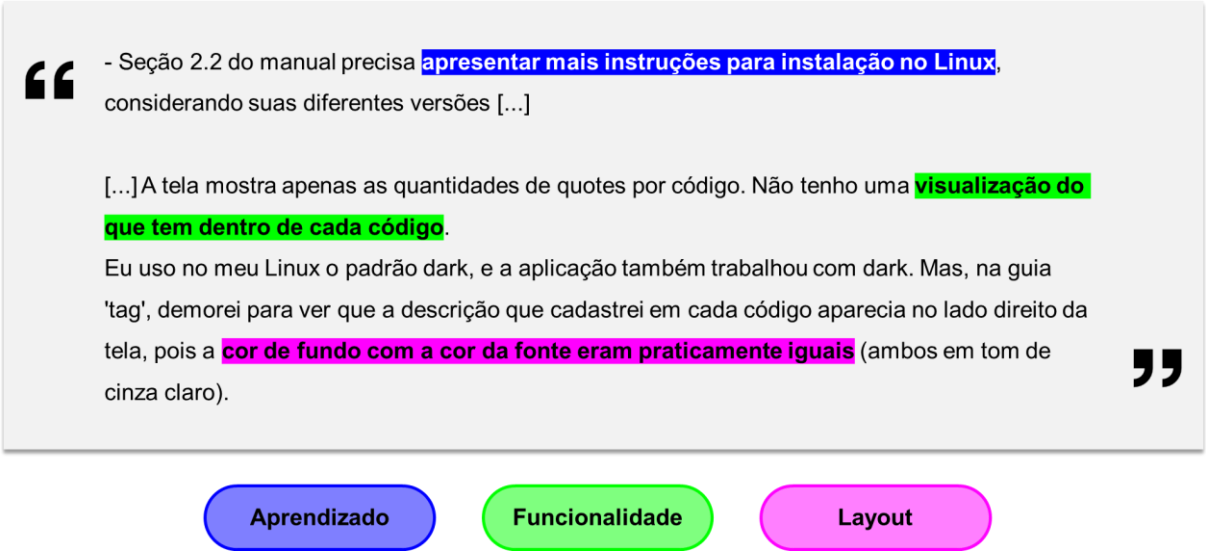
De forma semelhante, foi realizado um estudo qualitativo com base em três questões dissertativas, aplicadas aos usuários. As perguntas foram elaboradas com um escopo amplo, com o objetivo de coletar as perspectivas e opiniões dos participantes de maneira mais detalhada, contribuindo para o aprimoramento do software. As questões dissertativas opcionais incluídas nos questionários foram:

a) “Existem funcionalidades ou características que você sugeriria para melhorar o software?”;
b) “Como você avaliaria sua experiência geral com o software?”;
c) “Você gostaria de adicionar algum comentário sobre o produto ou suas impressões gerais sobre o projeto?”.

Na análise dos resultados obtidos, utilizou-se o próprio software desenvolvido, aplicando uma codificação fechada aos dados. Foram definidas três categorias de *tags* para classificar os pontos de melhoria: “**Funcionalidade”**, que abrange funcionalidades ausentes ou faltantes no sistema e na camada lógica; “**Aprendizado**”, que aborda dificuldades enfrentadas pelos usuários quanto ao entendimento da plataforma ou à aplicação dos recursos; e, por fim, “***Layout***”, que foca em sugestões de melhorias relacionadas à interface visual do produto.

Para exemplificar, um trecho da resposta 1 para a terceira questão foi extraído e está apresentado na Figura 23. Nele, é possível identificar os três aspectos mencionados, demonstrando como esses elementos foram abordados pelo usuário. A partir dessa análise, é possível extrair novos requisitos que podem ser incorporados em futuras iterações do software.

Figura 23 - Exemplo de codificação



Fonte: Elaborado pelos autores.

Assim, observou-se um equilíbrio na distribuição das tags ao longo das demais respostas, com 25 *tags* relacionadas a “Funcionalidade”, 24 *tags* associadas a “Aprendizado” e 23 *tags* referentes ao “*Layout*”.

Analisando mais detalhadamente os relatórios gerados pelo software, foi possível observar as nuvens de palavras associadas a cada um dos códigos. Para o código “Aprendizado”, destacaram-se termos de alta frequência como “tutoriais”, “entender”, “passo” e “exemplos”, conforme ilustrado na Figura 24.

De maneira semelhante, a *tag* “Funcionalidade”, que representa os requisitos ausentes na camada de lógica de negócios, evidenciou termos como “código”, *“tag”* (ambos referindo-se ao mesmo conceito), além de outros termos relevantes como “processo”, “visualização” e “codificação”, apresentados na Figura 25.

Figura 24 - Nuvem de palavras relacionadas a "Aprendizado"



Fonte: Elaborado pelos autores.

Figura 25 - Nuvem de palavras relacionadas a "Funcionalidades"



Fonte: Elaborado pelos autores.

Por fim, a nuvem de palavras associada à *tag* “*Layout*” destacou termos como” tela”, “interface”, “código” e “guia”, conforme mostra a Figura 26. Com isso, é possível realizar algumas análises e discussões acerca de melhorias para os próximos estágios.

Figura 26 - Nuvem de palavras relacionadas a "*layout*"



Fonte: Elaborado pelos autores.

6 DISCUSSÃO

Baseado nos resultados obtidos, foram identificados tanto pontos de melhoria quanto acertos relevantes no software desenvolvido. Nesta seção, será discutido o estado atual do projeto, avaliando até que ponto os objetivos propostos foram alcançados e quais aspectos podem ser aprimorados no futuro.

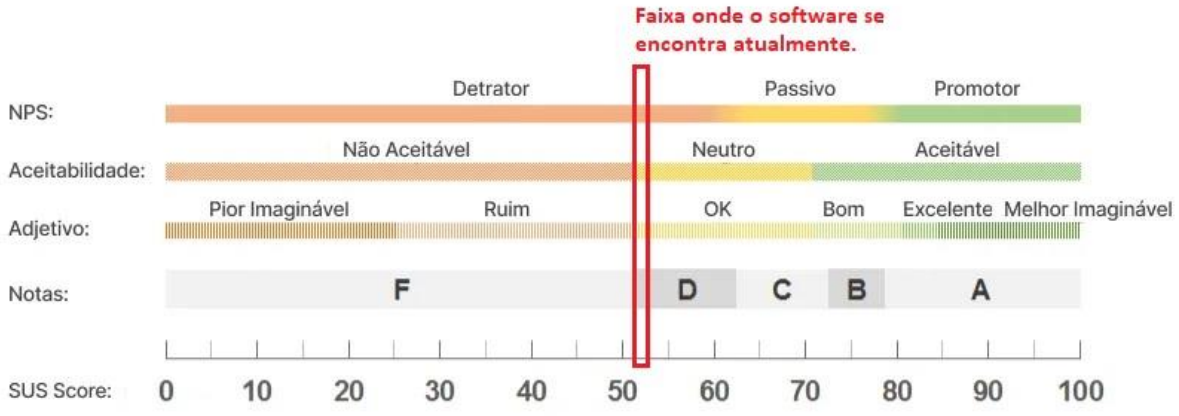
6.1 O SOFTWARE NA ESCALA SUS E NPS

Os primeiros pontos a serem discutidos referem-se às pontuações obtidas pelo software nas escalas SUS e NPS, que foram de 51,5 e -50, respectivamente. Esses resultados indicam um desempenho inicial mediano, refletindo a percepção dos usuários sobre a usabilidade e a satisfação com a ferramenta.

No caso da escala SUS, esse desempenho é corroborado pelos gráficos apresentados, nos quais a distribuição das respostas tende a se concentrar em torno da nota 3 na maioria das questões, sugerindo uma avaliação moderada. Já no NPS, de forma semelhante, a maior frequência de respostas se deu na nota 7, que indica uma postura neutra dos usuários. Esse comportamento contribuiu para que avaliações mais baixas influenciassem negativamente a pontuação final do *Net Promoter Score*.

Com isso, é possível definir a faixa onde o software se encontra atualmente. Conforme o comparativo da Figura 27, a partir das notas obtidas, o sistema apresenta uma aceitabilidade neutra, podendo ser associado com o adjetivo “*ok*”.

Figura 27 – Notas e adjetivos associadas aos escores brutos do SUS



Fonte: Adaptado de Barros (2022).

Também é possível avaliar o desempenho do software a partir de cada heurística analisada. Em relação à facilidade de aprendizagem, o software demonstrou ser, em geral, de fácil aprendizado. Isso é evidenciado pela maior frequência de notas 4 na questão 3, que avalia diretamente o quão simples foi utilizar o sistema. Ainda nessa heurística, a questão 4 revelou que as maiores dificuldades surgiram principalmente entre usuários sem experiência prévia com esse tipo de análise, o que os levou a considerar assistência externa para o uso do software.

Ao analisar a eficiência, observa-se que as notas apresentam uma tendência de centralidade, sugerindo que o sistema é capaz de cumprir suas funções conforme o proposto inicialmente, no entanto, ainda não atinge um nível de excelência, permanecendo em uma zona de neutralidade. Isso revela que, embora o sistema seja funcional, há espaço para aprimoramentos a fim de tornar sua operação mais eficiente e refinada.

Essa tendência de centralidade se repete nas heurísticas de facilidade de memorização e minimização de erros, indicando que o sistema, embora funcional, não se destaca nesses aspectos. No entanto, ao analisar a satisfação geral dos usuários, percebe-se que essa neutralidade impacta negativamente as notas de satisfação, especialmente quando questionados sobre a probabilidade de uso frequente do sistema. Essa insatisfação é refletida diretamente na escala NPS, que mostrou uma alta concentração de detratores.

Os resultados evidenciam, portanto, oportunidades claras de melhoria que podem aumentar tanto o engajamento quanto a adoção do software. Diante disso, podemos responder a duas das perguntas da nossa investigação:

"O software atende de forma eficaz às necessidades de seus usuários?" – A análise revela que o software cumpre suas funções básicas e atende às necessidades propostas, como evidenciado pelas pontuações centrais nas avaliações de eficiência. No entanto, o desempenho está longe de ser excelente, com o sistema operando em uma zona de neutralidade. Isso indica que, embora os usuários consigam realizar as tarefas propostas, o software ainda não oferece uma experiência suficientemente fluida ou otimizada, o que impacta sua eficácia geral.

"Como os usuários percebem a curva de aprendizado e a facilidade de uso do software?" – Com relação à esse aspecto, os resultados indicam que o

software é relativamente fácil de aprender, conforme demonstrado pela maior frequência de notas positivas na questão relacionada à simplicidade de uso (questão 3). No entanto, os usuários sem experiência prévia com esse tipo de análise ainda enfrentaram dificuldades, o que sugere que, embora o sistema seja acessível para alguns, há barreiras para aqueles sem familiaridade com a metodologia aplicada. Isso reflete uma curva de aprendizado suave para usuários mais experientes, mas desafiadora para iniciantes.

A última pergunta da investigação buscou ser respondida através das questões e com os dados qualitativos obtidos.

6.2 AS RESPOSTAS QUALITATIVAS

Conforme mencionado anteriormente, as respostas foram codificadas para a análise qualitativa utilizando a própria ferramenta desenvolvida, demonstrando sua eficácia parcial em atender à proposta e gerar insights úteis aos usuários. Os três códigos escolhidos — "aprendizado", "funcionalidade" e "*layout*" — revelaram aspectos importantes sobre os requisitos ainda faltantes para o software.

Na nuvem de palavras gerada para o código "aprendizado", surgiram termos como "tutoriais", "entender" e "passo" (possivelmente relacionado à expressão "passo a passo"). Esses termos indicam a necessidade de um manual de uso mais detalhado, com mais tutoriais e exemplos práticos sobre a utilização do software, desde a instalação até o uso de suas funcionalidades.

Em relação à "funcionalidade", os usuários destacaram termos como "visualização" e "processo". O termo "visualização" refere-se à forma como as informações são apresentadas aos usuários, sugerindo que a interface poderia exibir os dados de maneira mais clara ao longo do processo de codificação. Por exemplo, a funcionalidade de "visualizar o conteúdo de cada código" foi apontada como uma melhoria necessária, conforme é citado no exemplo do capítulo 5.2. Já o termo "processo" relaciona-se à demanda por funcionalidades que tornem o processo de codificação mais ágil, como a sugestão automática de rótulos baseada em algoritmos de aprendizado de máquina. No entanto, essa funcionalidade ficou fora do escopo da versão atual devido à sua complexidade.

Por fim, embora o design da interface não tenha sido o foco principal desta

versão do software, algumas respostas apontaram para a necessidade de revisões no "layout". Termos como "interface" e "tela" sugerem a necessidade de maior fluidez na operação, enquanto "responsividade" destaca as limitações da biblioteca utilizada para o desenvolvimento da interface gráfica (Tkinter), que carece de suporte robusto para recursos como a responsividade.

A partir dessas observações, é possível responder à última questão da investigação: **"Quais aspectos da interface e das funcionalidades precisam ser aprimorados?"** – Os principais pontos de melhoria incluem a necessidade de uma visualização mais clara durante o processo de codificação e a inclusão de funcionalidades, como a sugestão automática de rótulos. No que diz respeito à interface, é essencial aprimorar a fluidez e a responsividade, superando as limitações da biblioteca Tkinter. Graças à arquitetura em três camadas adotada no desenvolvimento, há flexibilidade para até mesmo mudar a linguagem da interface, garantindo que essa possibilidade de melhoria seja feita sem um esforço geral.

Dessa forma, identificam-se os requisitos que poderão ser incorporados nas próximas iterações do Ciclo de Vida de Desenvolvimento de Software (SDLC) do nosso projeto. Entre os requisitos futuros, destacam-se:

a) Melhor visualização dos códigos e trechos de dados;
b) Sugestão automática de rótulos com aprendizado de máquina;
c) Manual de uso mais detalhado, com tutoriais práticos;
d) Substituição da biblioteca Tkinter por uma interface mais moderna e responsiva;
e) Integração com outras ferramentas de análise qualitativa.

6.3 LIMITAÇÕES

Diante dos resultados e análises apresentados, reconhece-se que o software ainda não atingiu o grau de maturidade necessário para uma distribuição em larga escala. Contudo, essa possibilidade já havia sido prevista desde a concepção do projeto. A escolha de um SDLC iterativo permite justamente a revisão e aprimoramento contínuo dos requisitos, permitindo que cada iteração traga melhorias significativas ao software.

A seleção dos requisitos a serem trabalhados em cada iteração depende

diretamente de fatores como o tamanho da equipe, a disponibilidade de recursos e o tempo dedicado ao projeto — aspectos que, neste trabalho, foram limitados. Essas restrições influenciaram diretamente o ritmo de desenvolvimento e o alcance das funcionalidades implementadas na versão inicial.

Apesar desses desafios, o software apresentou notas medianas nas avaliações, o que indica que, embora longe da excelência, ele cumpre os requisitos básicos estabelecidos no planejamento. A pontuação moderada nas escalas de usabilidade e satisfação sugere que o sistema atende, de forma funcional, às necessidades dos usuários, mas ainda precisa de melhorias significativas.

Essas limitações podem ser gradualmente mitigadas conforme o software passa por novas iterações do ciclo de desenvolvimento. À medida que mais recursos sejam alocados e mais *feedback* dos usuários seja coletado, o sistema poderá evoluir, ganhando robustez e refinamento necessários para alcançar um nível de maturidade adequado para uma adoção mais ampla.

7 CONSIDERAÇÕES FINAIS

Com o aumento exponencial no volume de dados gerados, a análise de informações qualitativas se torna cada vez mais comum para diversas áreas do conhecimento. Esses dados, por sua natureza não estruturada e subjetiva, requerem técnicas específicas que possam extrair *insights* significativos. Dentre essas técnicas, a codificação se destaca como uma abordagem central, permitindo que padrões e conceitos emergentes sejam identificados e organizados de maneira sistemática. Para apoiar esse processo, os softwares de análise assistida por computador, conhecidos como CAQDAS, desempenham um papel fundamental, agilizando e estruturando o trabalho dos pesquisadores.

Este trabalho focou, portanto, no desenvolvimento de um software dedicado à codificação de dados qualitativos, buscando preencher lacunas importantes observadas no mercado atual de CAQDAS, como o alto custo das ferramentas comerciais e a escassez de soluções robustas em código aberto. O objetivo principal foi criar uma alternativa acessível e eficiente, que democratizasse o uso dessas ferramentas, permitindo que mais pesquisadores tivessem acesso a uma plataforma de codificação de dados sem os altos custos das soluções proprietárias.

A metodologia de desenvolvimento adotada seguiu o modelo de SDLC iterativo, em que as fases de análise, design, implementação e testes são continuamente repetidas, permitindo melhorias graduais e sucessivas ao longo do ciclo de vida do software. Esse processo iterativo garantiu a flexibilidade necessária para incorporar *feedbacks* e realizar ajustes progressivos à medida que o software evolui. A arquitetura, por sua vez, foi a de três camadas, separando a interface do usuário, a lógica de negócios e o armazenamento de dados. Essa escolha técnica proporcionou independência entre os componentes do sistema, favorecendo a modularidade, manutenção e futuras expansões, além de facilitar ajustes sem a necessidade de reestruturar toda a aplicação.

Dessa forma, a avaliação da primeira versão do software revelou que, apesar de atender aos requisitos iniciais de funcionalidade, a taxa de aceitabilidade entre os usuários ficou em um nível neutro. Esse resultado reflete que, embora o software tenha potencial, ele ainda não está totalmente pronto para ser amplamente utilizado

ou divulgado para o público geral. Melhorias significativas são necessárias, especialmente no que diz respeito a funcionalidades mais robustas e à interface gráfica, que foram identificadas como pontos críticos a serem aperfeiçoados. Contudo, essas melhorias podem ser implementadas de forma eficiente, uma vez que a separação em camadas permite a reformulação da interface sem comprometer a lógica de negócios ou o armazenamento de dados.

Além disso, a análise do *feedback* dos usuários destacou que o software atende apenas parcialmente ao seu objetivo principal: ser uma ferramenta acessível que facilite o processo de codificação de dados qualitativos. Para cumprir integralmente as expectativas do público-alvo, algumas necessidades devem ser atendidas, conforme os pontos acima mencionados.

Apesar dessas limitações, o projeto já alcançou um avanço importante ao oferecer uma alternativa viável e de baixo custo para a análise qualitativa. No entanto, ainda há um longo caminho a ser percorrido até que o software se estabeleça como uma solução completa no mercado de CAQDAS.

Por fim, o presente trabalho deixa claras as oportunidades para futuras iterações de desenvolvimento. Através de um modelo colaborativo, o software foi disponibilizado em um repositório público, incentivando a participação da comunidade científica e de desenvolvedores. As próximas etapas incluem a implementação de novos requisitos, como os mencionados no capítulo 6, e o aperfeiçoamento contínuo do sistema com base no *feedback* dos usuários. A expectativa é que, com o tempo e contribuições da comunidade, o software possa evoluir para se tornar uma ferramenta de referência para a análise qualitativa, atendendo tanto às demandas de cada público.

REFERÊNCIAS

ADU, Philip. **A Step-by-Step Guide to Qualitative Data Coding**. New York : Routledge, 2019.: Routledge, 2019. Disponível em: https://www.taylorfrancis.com/books/9781351044509.

ALBERT, Bill; TULLIS, Tom. **Measuring the User Experience: Collecting, Analyzing, and Presenting UX metrics**. Oxford, England: Morgan Kaufmann, 2022.

ALLAN, Graham; SKINNER, Chris (org.). **Handbook for Research Students in Social Sciences**. 1. ed. London: Routledge, 2020. Disponível em: https://www.taylorfrancis.com/books/9781000107869.

AMÉRICO, Bruno. **Método de pesquisa qualitativa: analisando fora da caixa a prática de pesquisar organizações**. Rio de Janeiro: Alta Books, 2021.

ARAGÃO, Thiago. **Solid — D.I.P — Dependency Inversion Principle**. 2019.

ASPERS, Patrik; CORTE, Ugo. What is Qualitative in Qualitative Research. **Qualitative Sociology**, v. 42, n. 2, p. 139–160, 2019.

ATLAS.TI. **A brief history of ATLAS.ti**. 2024. Disponível em: https://atlasti.com/about-atlas-ti. Acesso em: 1 de abril de 2024.

AWS. **O que é o SDLC (ciclo de vida de desenvolvimento de software)?**. 2024. Disponível em: https://aws.amazon.com/what-is/sdlc/. Acesso em: 27 set. 2024.

AYAR, Mehmet C.; YALVAC, Bugrahan. An ethnographic study of an engineering community: Mentoring as a tacit rule and its implications for how people learn. **Journal of Engineering Education**, v. 111, n. 2, p. 400–419, 2022.

BARROS, Myrela. **Guia atualizado de como utilizar a escala SUS (System Usability Scale) no seu produto**. 2022. Disponível em: https://brasil.uxdesign.cc/guia-atualizado-de-como-utilizar-a-escala-sus-system-usability-scale-no-seu-produto-ab773f29c522. Acesso em 30 de set. de 2024.

BLISS, Linda A. Phenomenological Research. **International Journal of Adult Vocational Education and Technology**, v. 7, n. 3, p. 14–26, 2016.

BRASIL, Eric; NASCIMENTO, Leonardo Fernandes. História Digital: Reflexões a partir da Hemeroteca Digital Brasileira e do uso de CAQDAS na reelaboração da pesquisa histórica. **Estudos Históricos (Rio de Janeiro)**, v. 33, n. 69, p. 196–219, 2020.

BROOKE, John. SUS: A quick and dirty usability scale. **Usability Eval. Ind.**, v. 189, 1995.

BYTHEWAY, Julie. Using grounded theory to explore learners' perspectives of workplace learning. **International Journal of Work-Integrated Learning**, v. 19, p. 249–259, 2018.

CORBIN, Juliet M; STRAUSS, Anselm C. **Basics of qualitative research**. 4. ed. Thousand Oaks, CA: SAGE Publications, 2015.

CORTI, Louise; GREGORY, Arofan. CAQDAS Comparability. What about CAQDAS Data Exchange?. **Forum: Qualitative Social Research**, v. 12, n. 1, 2011.

CURTAIN, Colin. **QualCoder**. 2023. Disponível em: https://github.com/ccbogel/QualCoder. Acesso em: 23 mar. 2024.

DELVE. **About Us**. 2024. Disponível em: https://delvetool.com. Acesso em: 25 maio 2024.

DETERDING, Nicole M.; WATERS, Mary C. Flexible Coding of In-depth Interviews: A Twenty-first-century Approach. **Sociological Methods & Research**, v. 50, n. 2, p. 708–739, 2021.

FLICK, Uwe. **The SAGE Handbook of Qualitative Data Collection**. London: SAGE Publications, 2018.

GAJDZIK, Bożena; WOLNIAK, Radosław. Digitalisation and Innovation in the Steel Industry in Poland—Selected Tools of ICT in an Analysis of Statistical Data and a Case Study. **Energies**, v. 14, n. 11, p. 3034, 2021.

GARG, A; POPLI, R; SARAO, B S. Growth of Digitization and its Impact on Big Data Analytics. **IOP Conference Series: Materials Science and Engineering**, v. 1022, n. 1, p. 012083, 2021.

GLASER, Barney G.; STRAUSS, Anselm L. **The discovery of grounded theory: strategies for qualitative research**. Aldine, 1999. 271 p.

GOLDENBERG, Mirian. **A arte de pesquisar: como fazer pesquisa qualitativa em ciências sociais**. 7. ed. Rio de Janeiro, RJ: Record, 2003. v. 1

GORTON, Ian. **Essential Software Architecture**. 2. ed. Berlin, Germany: Springer, 2011.

GROENEWALD, Thomas. A Phenomenological Research Design Illustrated. **International Journal of Qualitative Methods**, v. 3, n. 1, p. 42–55, 2004.

HO, LaiYee; LIMPAECHER, Alex. **How To Do Open, Axial and Selective Coding in Grounded Theory**. 2022. Disponível em: https://delvetool.com/blog/openaxialselective. Acesso em: 22 mar. 2024.

KAUFMANN, Andrea A; BARCOMB, Ann; RIEHLE, Dirk. Supporting Interview Analysis with Autocoding. *In*:, 2020. **Hawaii International Conference on System Sciences**. 2020.

LAPLANTE, Phillip A.; KASSAB, Mohamad. **What Every Engineer Should Know about Software Engineering**. Boca Raton: CRC Press, 2022.

LARMAN, C.; BASILI, V.R. Iterative and incremental developments. a brief history. **Computer**, v. 36, n. 6, p. 47–56, 2003.

LESTER, Jessica Nina; CHO, Yonjoo; LOCHMILLER, Chad R. Learning to Do Qualitative Data Analysis: A Starting Point. **Human Resource Development Review**, v. 19, n. 1, p. 94–106, 2020.

LIKERT, R. A technique for the measurement of attitudes. **Archives of Psychology**, v. 22 140, p. 55, 1932.

LÜDKE, Menga; ANDRÉ, Marli E. D. A. **Pesquisa em Educação - Abordagens Qualitativas.** 2. ed. São Paulo, SP: E.P.U., 2022.

MARQUES, Nuno; SILVA, Rodrigo Rocha; BERNARDINO, Jorge. Using ChatGPT in Software Requirements Engineering: A Comprehensive Review. **Future Internet**, v. 16, n. 6, p. 180, 2024.

MATTAR, João; RAMOS, Daniela Karine. **Metodologia da pesquisa em educação: abordagens qualitativas, quantitativas e mistas**. São Paulo: Almedina Brasil, 2021. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9786586618518/. Acesso em: 21 abr. 2024.

MOELLER, Giovanna. **O Princípio da Responsabilidade Única (Single Responsibility Principle — SRP) do SOLID**. 2024. Disponível em: https://giovannamoeller.medium.com/o-princípio-da-responsabilidade-única-single-responsibility-principle-srp-do-solid-21707ec45304. Acesso em: 27 set. 2024.

MOHAJAN, Devajit; MOHAJAN, Haradhan Kumar. Exploration of Coding in Qualitative Data Analysis: Grounded Theory Perspective. **Research and Advances in Education**, v. 1, n. 6, p. 50–60, 2022.

NIELSEN, Jakob. Enhancing the explanatory power of usability heuristics. *In*: , 1994, New York, New York, USA. **Proceedings of the SIGCHI conference on Human factors in computing systems celebrating interdependence - CHI '94**. New York, New York, USA: ACM Press, 1994. p. 152–158.

NISANSALA, Sewwandi *et al.* Microservice Based Edge Computing Architecture for Internet of Things. *In*:, 2022. **2022 2nd International Conference on Advanced Research in Computing (ICARC)**. IEEE, 2022. p. 332–337.

NODARI, Felipe *et al.* Contribuição do Maxqda e do NVivo para a Realização da Análise de Conteúdo. *In*: , 2014, Rio de janeiro / RJ. **XXXVIII Encontro da ANPAD**. Rio de janeiro / RJ: 2014. Disponível em: https://repositorio.pucrs.br/dspace/bitstream/10923/10235/2/Contribuicao_do_Maxqda_e_do_NVivo_para_a_Realizacao_da_Analise_de_Conteudo.pdf. Acesso em: 22 maio 2024.

OLAPANE, Elias C. An In-depth Exploration on the Praxis of Computer-assisted Qualitative Data Analysis Software (CAQDAS). **Journal of Humanities and Social Sciences Studies**, v. 3, n. 11, p. 57–78, 2021.

OLSEN, Wendy. **Coleta de dados: debates e métodos fundamentais em pesquisa social**. Porto Alegre: Penso, 2015.

QUTOSHI, Sadruddin Bahadur. Phenomenology: A Philosophy and Method of Inquiry. **Journal of Education and Educational Development**, v. 5, n. 1, p. 215, 2018.

RAHMAN, Md Shidur. The Advantages and Disadvantages of Using Qualitative and Quantitative Approaches and Methods in Language “Testing and Assessment” Research: A Literature Review. **Journal of Education and Learning**, v. 6, n. 1, p. 102, 2016.

REICHHELD, Fred; MARKEY, Rob. **The Ultimate Question 2.0 (Revised and Expanded Edition): How Net Promoter Companies Thrive in a Customer-Driven World**. Cambridge, MA: Harvard Business Review Press, 2011.

ROGERS, Marla. Coding Qualitative Data. *In*: OKOKO, Janet Mola; TUNISON, Scott,; WALKER, Keith D. (org.). **Varieties of Qualitative Research Methods**. Cham: Springer International Publishing, 2023. p. 73–78.

ROYCE, W W. Managing the development of large software systems: concepts and techniques. *In*:, 1987, Washington, DC, USA. **Proceedings of the 9th International Conference on Software Engineering**. Washington, DC, USA: IEEE Computer Society Press, 1987. p. 328–338.

SALDANA, Johnny M. **The coding manual for qualitative researchers**. 2. ed. London: SAGE Publications, 2012.

SANGASUBANA, Nisaratana. How to Conduct Ethnographic Research. **The Qualitative Report**, v. 16, n. 2, 2014.

SCHOONENBOOM, Judith. The Fundamental Difference Between Qualitative and Quantitative Data in Mixed Methods Research. **Forum: Qualitative Social Research**, v. 24, n. 1, 2023. Disponível em: https://doi.org/10.17169/fqs-24.1.3986. Acesso em: 20 abr. 2024.

SFORZA, Guilherme Santini; DOS SANTOS, Luiz Felipe Looze. **Análise de sentimentos em textos em português do Brasil**. 2023. 51 f. Trabalho de Conclusão de Curso (Graduação) - Centro Universitário Facens, Sorocaba, SP, 2023. Disponível em: https://pergamum.facens.br/acervo/24423. Acesso em: 18 abr. 2024.

SHVETS, Alexander. **Dive Into Design Patterns**. 1. ed., 2021. v. 1

SILVA, Dalila. **O Guia do NPS: o que a Net Promoter Score tem a dizer sobre sua empresa?**. 2020. Disponível em: https://www.rdstation.com/blog/marketing/o-que-e-nps/. Acesso em: 27 ago. 2024.

SINEK, Simon. **Comece pelo porquê: Como grandes líderes inspiram pessoas e equipes a agir**. Sextante, 2018.

SKJOTT LINNEBERG, Mai; KORSGAARD, Steffen. Coding qualitative data: a synthesis guiding the novice. **Qualitative Research Journal**, v. 19, n. 3, p. 259–270, 2019.

SOMMERVILLE, Ian. **Engineering software products**. Upper Saddle River, NJ: Pearson, 2019.

STRAUSS, Anselm; CORBIN, Juliet. **Basics of qualitative research: Techniques and procedures for developing grounded theory**. 2. ed. Thousand Oaks, CA, US: Sage Publications, Inc, 1998.

TKACHENKO, Maxim *et al.* **Label Studio: Data labeling software**. 2022. Disponível em: https://github.com/heartexlabs/label-studio. Acesso em: 23 mar. 2024.

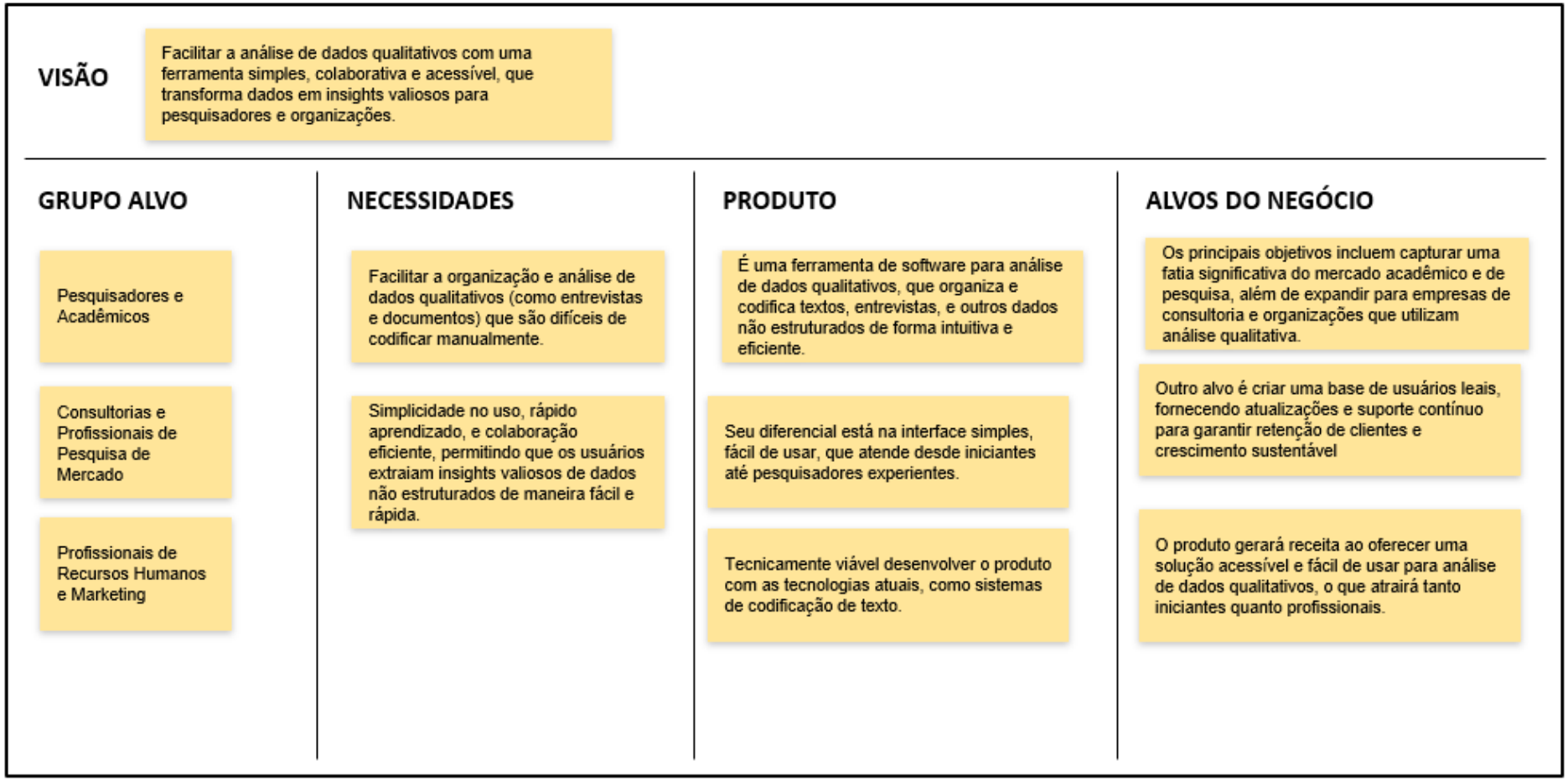
TULLIS, Tom; ALBERT, Bill. Combined and Comparative Metrics. *In*: MEASURING USER EXPERIENCE. Elsevier, 2013. p. 187–208.

TUTORIALSPOINT. **SDLC - Iterative Incremental Model**. 2024. Disponível em: https://www.tutorialspoint.com/adaptive_software_development/sdlc_iterative_incremental_model.htm. Acesso em: 27 set. 2024.

VIANNA, Bernardo. **O mar de dados virou um oceano e não para de crescer. Mas nem tudo é aproveitado**. 2021. Disponível em: https://www.insper.edu.br/noticias/o-mar-de-dados-virou-um-oceano-e-nao-para-de-crescer-mas-nem-tudo-e-aproveitado/. Acesso em: 28 mar. 2024.

WILLIAMS, Michael; MOSER, Tami. The Art of Coding and Thematic Exploration in Qualitative Research. **International Management Review**, v. 15, p. 45, 2019.

APÊNDICE A – *PRODUCT VISION BOARD*



APÊNDICE B – *STAKEHOLDERS* IDENTIFICADOS

	Descrição
ID	STK1
Nome	Pesquisador Acadêmico
Função	Utiliza o software para análise de dados qualitativos em sua pesquisa acadêmica.
Exp. no assunto	experiência em análise de dados qualitativos
Exp. tecnológica	Conhecimento intermediário em informática, familiaridade com softwares similares como NVivo e MaxQDA
Características de influência	Necessidade de ferramentas avançadas de codificação, capacidade de gerar relatórios personalizados para publicação acadêmica
Prioridade	Alta, pois sua satisfação e adoção do software podem impactar sua produtividade e credibilidade acadêmica

	Descrição
ID	STK2
Nome	Desenvolvedor de Software
Função	
Exp. no assunto	Experiência em desenvolvimento de software
Exp. tecnológica	Conhecimento avançado em programação, familiaridade com frameworks e linguagens relevantes para o projeto.
Características de influência	Conhecimento das necessidades técnicas e limitações do projeto, capacidade de implementar funcionalidades de forma eficiente.
Prioridade	Alta, pois sua capacidade de atender aos requisitos técnicos e de desenvolvimento afeta diretamente a qualidade e a competitividade do software

	Descrição
ID	STK3
Nome	Estudante de Graduação/Pós-Graduação
Função	Utiliza o software para análise de dados em sua dissertação ou trabalhos acadêmicos
Exp. no assunto	Graduação em Psicologia, iniciante em pesquisa qualitativa
Exp. tecnológica	Conhecimento básico em informática, pouca ou nenhuma experiência prévia com softwares de análise de dados
Características de influência	Interface intuitiva, suporte técnico acessível, recursos de aprendizado integrados
Prioridade	Média, pois sua satisfação pode influenciar futuras escolhas de software

	Descrição
ID	STK4
Nome	Analista de Dados
Função	Utiliza o software para análise de dados qualitativos em uma empresa de pesquisa de mercado
Exp. no assunto	Graduação em Estatística, experiência em análise de dados por mais de 3 anos.
Exp. tecnológica	Conhecimento intermediário a avançado em informática, experiência com ferramentas de análise de dados.
Características de influência	Necessidade de integração com outras ferramentas de análise de dados e geração de relatórios automatizados.
Prioridade	Alta, pois sua eficiência afeta diretamente a capacidade do software de fornecer insights aos clientes

APÊNDICE C – REQUISITOS DO SISTEMA

Requisitos funcionais do sistema:

ID	RF001	Versão	1.0
Nome	Importação de Dados		
Descrição	O sistema deve permitir que os usuários importem dados de suas pesquisas em uma variedade de formatos, como arquivos de texto. Inicialmente o sistema deve suportar apenas formatos de texto.		
Autor	Equipe de Desenvolvimento		
Fonte	Brainstorm de Requisitos		
Importância	Essencial para facilitar o processo de entrada de dados, permitindo que os pesquisadores integrem facilmente seus materiais de pesquisa ao sistema de análise.		
Critério de aceite	O sistema deve suportar a importação de arquivos nos formatos TXTe PDF, garantindo a integridade dos dados e sua pronta acessibilidade para análise após a importação.		
Estabilidade	Estável		
Criticidade	Alta		
Prioridade	Necessário		

ID	RF002	Versão	1.0
Nome	Codificação de Dados		
Descrição	O sistema deve fornecer uma interface intuitiva e flexível para permitir que os usuários realizem a codificação dos dados, identificando temas, categorias e relações entre elementos. Isso inclui a capacidade de criar códigos personalizados, atribuí-los a trechos de dados e organizar hierarquicamente os códigos para uma análise mais detalhada.		
Autor	Equipe de Desenvolvimento		
Fonte	Brainstorm de Requisitos		
Importância	Fundamental para o processo de análise de dados qualitativos, permitindo que os pesquisadores organizem e interpretem as informações de maneira significativa.		
Critério de aceite	Os usuários devem poder criar códigos de forma rápida e fácil, atribuí-los a trechos de dados por meio de uma interface de arrastar e soltar, e visualizar as relações entre códigos e dados codificados para uma análise mais aprofundada.		
Estabilidade	Estável		

Criticidade	Alta
Prioridade	Necessário

ID	RF003	Versão	1.0
Nome	Análise de Frequência de Códigos		
Descrição	O sistema deve ser capaz de realizar análises de frequência dos códigos atribuídos aos dados, mostrando a ocorrência de cada código ao longo do conjunto de dados. Isso inclui a geração de relatórios detalhados que destacam os códigos mais frequentes e sua distribuição ao longo do tempo e do contexto da pesquisa.		
Autor	Equipe de Desenvolvimento		
Fonte	Brainstorm de Requisitos		
Importância	Essencial para identificar padrões e tendências nos dados, fornecendo insights valiosos para os usuários e orientando o desenvolvimento de hipóteses e conclusões.		
Critério de aceite	Os usuários devem poder gerar relatórios que mostrem a frequência de cada código, com opções de filtragem por período de tempo, contexto e outras variáveis relevantes para a pesquisa.		
Estabilidade	Estável		
Criticidade	Alta		
Prioridade	Requerido		

ID	RF004	Versão	1.0
Nome	Gerar Relatórios		
Descrição	O sistema deve permitir aos usuários gerar relatórios personalizados com base nos dados codificados e nas análises realizadas. Os relatórios devem ser configuráveis e podem incluir estatísticas de frequência de códigos, gráficos de distribuição de códigos ao longo do tempo, e insights qualitativos derivados da análise.		
Autor	Equipe de Desenvolvimento		
Fonte	Brainstorm de Requisitos		
Importância	Fundamental para fornecer uma ferramenta completa de análise de dados, permitindo que os usuários comuniquem os resultados de suas análises de forma clara e eficaz.		
Critério de aceite	Os usuários devem poder selecionar os parâmetros de relatório desejados, como período de tempo, códigos específicos a serem incluídos e formato de saída. O sistema deve então gerar o relatório de acordo com as		

	preferências do usuário e disponibilizá-lo para download ou visualização.
Estabilidade	Estável
Criticidade	Média
Prioridade	Requerido

ID	RF005	Versão	1.0
Nome	Salvar o Projeto		
Descrição	O sistema deve permitir aos usuários salvarem o estado atual do projeto de análise, incluindo os dados importados, os códigos atribuídos e quaisquer análises realizadas. Isso permite que os usuários interrompam o trabalho e retomem de onde pararam em uma sessão posterior.		
Autor	Equipe de Desenvolvimento		
Fonte	Brainstorm de Requisitos		
Importância	Essencial para garantir a continuidade do trabalho e evitar a perda de dados ou progresso durante o processo de análise de dados, especialmente em projetos de longo prazo.		
Critério de aceite	Os usuários devem poder salvar o projeto atual em um formato de arquivo específico do sistema. O sistema deve manter a integridade dos dados salvos e permitir que os usuários restaurem o projeto a partir do arquivo salvo a qualquer momento.		
Estabilidade	Estável		
Criticidade	Alta		
Prioridade	Necessário		

APÊNDICE D – CASOS DE USO

Nome	Abrir Projeto
Descrição	Este caso de uso descreve como um usuário pode abrir um novo projeto ou um projeto existente no sistema.
Agente	Usuário
Evento inicial	O usuário deseja iniciar um novo projeto ou continuar o trabalho em um projeto já existente.
Dados de entrada	Nome do projeto ou caminho do arquivo existente.
Pré-condição	O sistema está disponível e o usuário tem permissões adequadas para abrir o projeto.
Pós-condição	O projeto é aberto com sucesso no sistema.
Resultado	O usuário pode começar ou continuar a análise no projeto aberto.
Passos essenciais	O usuário seleciona a função de abrir projeto no sistema.
	O usuário escolhe entre abrir um novo projeto ou um projeto existente.
	(include) Se for um projeto existente, o usuário procura o arquivo correspondente.
	O sistema carrega e abre o projeto para uso.
Cenários alternativos	Se o arquivo do projeto existente não for encontrado, o sistema notifica o usuário.

Nome	Codificar Texto
Descrição	Este caso de uso descreve como o usuário pode codificar trechos de texto com base em categorias ou tags definidas.
Agente	Usuário
Evento inicial	O usuário identifica um trecho de texto que precisa ser codificado.
Dados de entrada	Trecho de texto e tags a serem aplicadas.
Pré-condição	O projeto está aberto.
Pós-condição	O texto é codificado com as tags selecionadas.
Resultado	O usuário pode analisar o texto com base nas tags aplicadas.
Passos essenciais	(Extend) O usuário cria uma questão e adiciona as respostas, se necessário.
	O usuário seleciona o trecho de texto no sistema.
	(Extend) O usuário cria tags específicas para codificar, se necessário.
	O usuário escolhe as tags apropriadas para associar ao trecho de texto.
	O sistema aplica as tags ao trecho de texto.
Cenários	

alternativos	

Nome	Criar Questão
Descrição	Este caso de uso descreve como um usuário pode criar uma nova questão no sistema para análise.
Agente	Usuário
Evento inicial	O usuário decide que uma nova questão de análise é necessária.
Dados de entrada	Descrição da questão.
Pré-condição	O projeto está aberto e o usuário tem permissões adequadas para criar questões.
Pós-condição	A questão é criada e está disponível para análise no projeto.
Resultado	O usuário pode usar a nova questão para orientar a análise de texto.
Passos essenciais	O usuário acessa a função de criar questões no sistema.
	O usuário insere a descrição da questão.
	(Include) O usuário adiciona respostas para aquela questão.
	O sistema confirma a criação da nova questão.
Cenários alternativos	Se os arquivos de reposta não estiverem no formato correto, não é possível criar as respostas.

Nome	Criar Tag
Descrição	Este caso de uso descreve como um usuário pode criar uma nova tag para categorizar trechos de texto durante a análise.
Agente	Usuário
Evento inicial	O usuário identifica a necessidade de uma nova categoria para organizar trechos de texto.
Dados de entrada	Nome e descrição da nova tag.
Pré-condição	O projeto está aberto e o usuário tem permissões adequadas para criar novas tags.
Pós-condição	A nova tag é criada e disponibilizada para uso durante a codificação.
Resultado	O usuário pode atribuir a nova tag aos trechos de texto relevantes durante a análise.
Passos essenciais	O usuário acessa a função de criação de tags no sistema.
	O usuário fornece um nome e uma descrição para a nova tag.
	O sistema confirma a criação da nova tag e a disponibiliza para uso.
Cenários alternativos	Se o nome da tag já estiver em uso, o sistema solicita que o usuário escolha um nome diferente ou modifique a descrição.

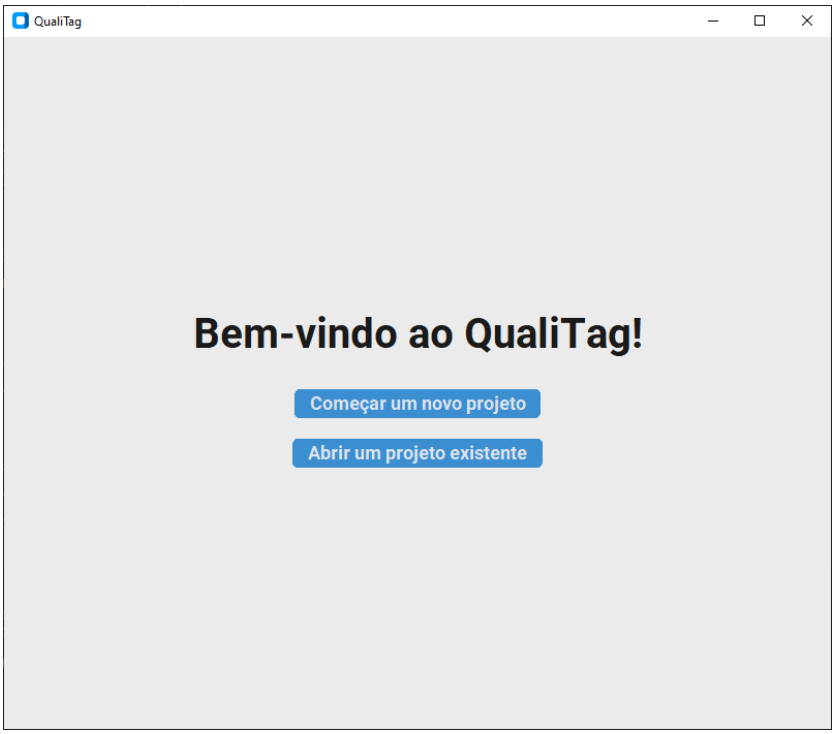
Nome	Gerar Relatórios
Descrição	Este caso de uso descreve como o usuário pode gerar relatórios a partir dos dados codificados.
Agente	Usuário
Evento inicial	O usuário deseja obter relatórios da análise realizada.
Dados de entrada	Opções de caminho e formato do relatório (PDF, Excel).
Pré-condição	Os dados de análise estão prontos para serem processados em relatórios.
Pós-condição	O relatório é gerado no formato desejado.
Resultado	O usuário tem o relatório gerado para consulta e análise.
Passos essenciais	O usuário acessa a função de geração de relatórios.
	(Include) O usuário seleciona o caminho e formato do relatório.
	O sistema gera o relatório e o disponibiliza para o usuário.
Cenários alternativos	Se houver falha na geração do relatório, o sistema notifica o usuário.

Nome	Salvar Projeto
Descrição	Este caso de uso descreve como o usuário pode salvar o projeto em andamento.
Agente	Usuário
Evento inicial	O usuário deseja salvar as alterações feitas no projeto.
Dados de entrada	Nome do arquivo e localização para salvar o projeto.
Pré-condição	O projeto está aberto e o usuário fez alterações que precisam ser salvas.
Pós-condição	O projeto é salvo no local especificado.
Resultado	O usuário tem o projeto salvo para futuras consultas ou edições.
Passos essenciais	O usuário acessa a função de salvar o projeto no sistema.
	(Include) O usuário escolhe a localização e o nome do arquivo para salvar.
	O sistema salva o projeto no local especificado.
Cenários alternativos	Se o local de salvamento já houver um arquivo com o mesmo nome, o sistema solicita confirmação para substituir o arquivo existente.

APÊNDICE E – APRESENTAÇÃO E TUTORIAL DE USO DO SOFTWARE

Quando você executar o software pela primeira vez, verá uma tela inicial com dois botões, um para iniciar um novo projeto e outro para continuar um projeto salvo, conforme mostra a Figura 28. Os projetos salvos estão em formato **“.pkl”**. Para a primeira vez usando o software, você deverá iniciar um novo projeto.

Figura 28 - Tela de abertura do software



Fonte: Elaborado pelos autores.

Assim que abrir, ou iniciar, um projeto você verá a aba inicial, chamada “Projeto” (Figura 29), nela você poderá adicionar novas questões aos projetos, no botão azul indicado.

Ao clicar no botão, uma janela *pop-up* será aberta. Nela haverá um campo para preencher qual foi a questão, ou pergunta realizada, que gerou aquelas respostas (Pensando respostas de uma entrevista, aqui poderia ser a questão que o entrevistador fez). Também haverá uma opção para selecionar a pasta onde se encontram as respostas, apresentado na Figura 30.

Importante: é necessário que os diretórios estejam devidamente organizados – 1 diretório por questão, onde dentro estará os arquivos de texto em PDF de cada

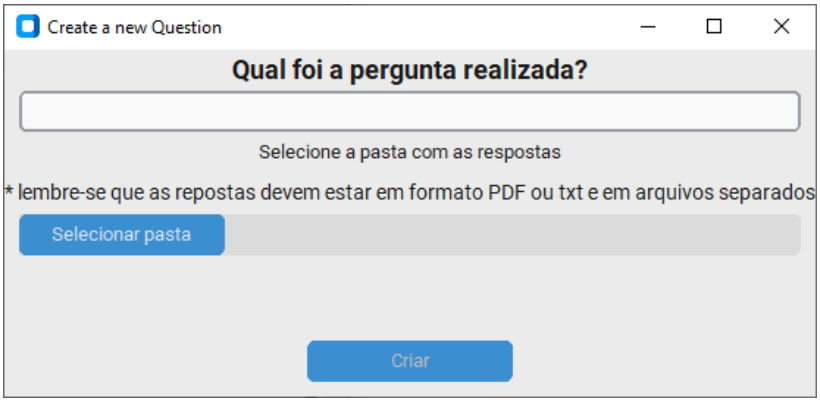
resposta. Cada resposta deve estar em um arquivo diferente.

Figura 29 – Aba “projeto” do software



Fonte: Elaborado pelos autores.

Figura 30 - Janela de adicionar questão

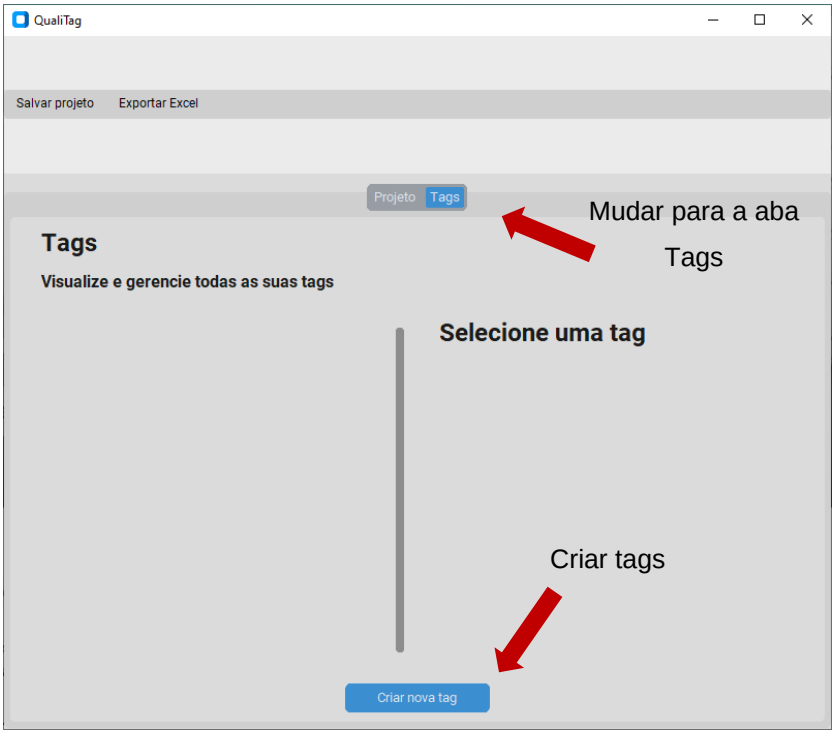


Fonte: Elaborado pelos autores.

As *tags* podem ser visualizadas, criadas ou deletadas na aba “*Tags*”, você pode navegar até elas pelo menu superior. Nessa página você encontrará uma lista com os códigos já criados, e um botão “X” para deletá-las. Ao clicar no nome de alguma *tag*, você poderá ver a sua descrição (Figura 31).

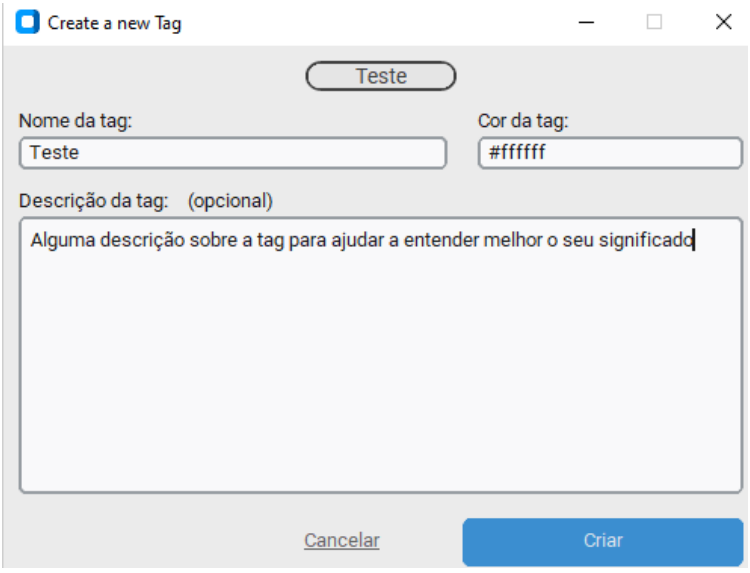
Ao fim, há um botão para a criação de novas *tags*, que abrirá uma nova janela similarmente como a Figura 32.

Figura 32 – Aba “*tags*” do software



Fonte: Elaborado pelos autores.

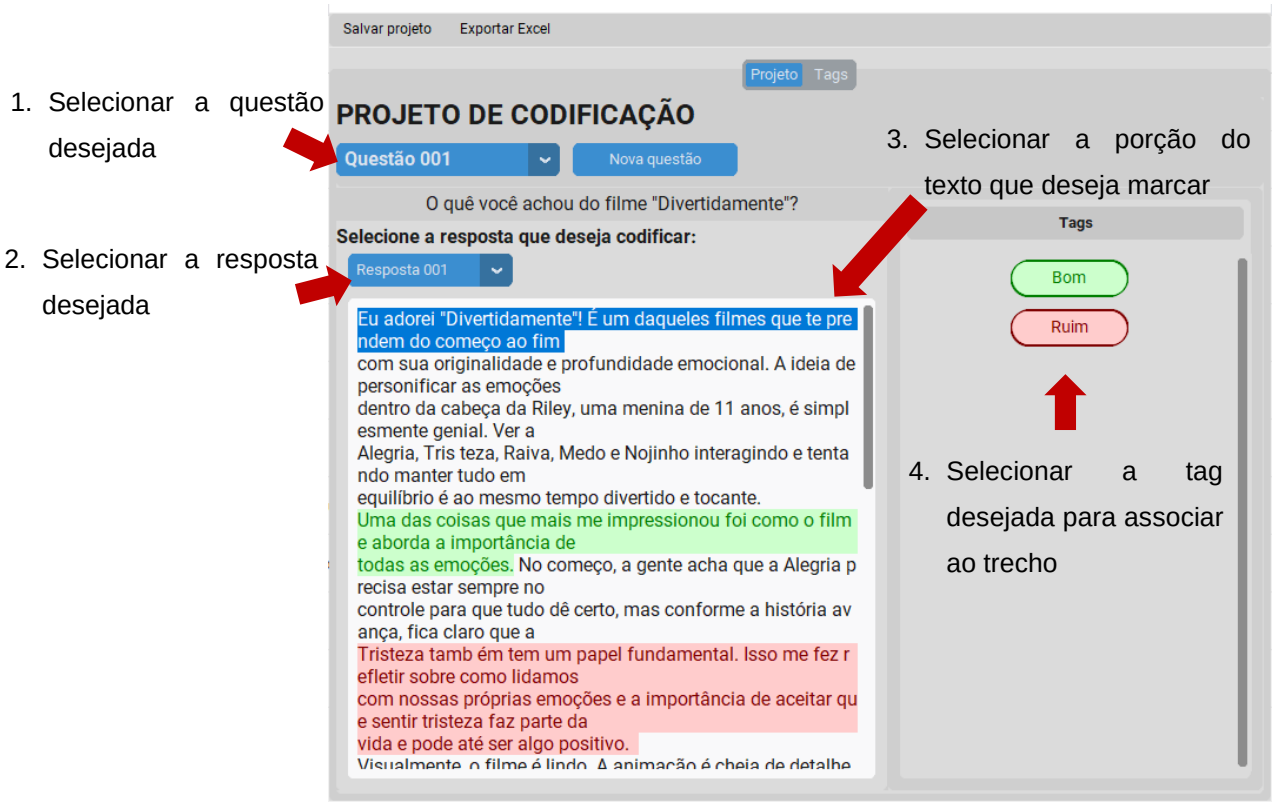
Figura 31 – Janela de criação de *tags*



Fonte: Elaborado pelos autores.

Com as *tags* criadas você já pode iniciar o processo de codificação. Para isso, basta voltar até a aba projeto e seguir os seguintes passos: Selecionar a questão desejada, selecionar qual das respostas deseja codificar, selecionar um trecho do texto e clicar na respectiva *tag* que deseja associas aquele texto. Esse processo pode ser visto conforme a Figura 33.

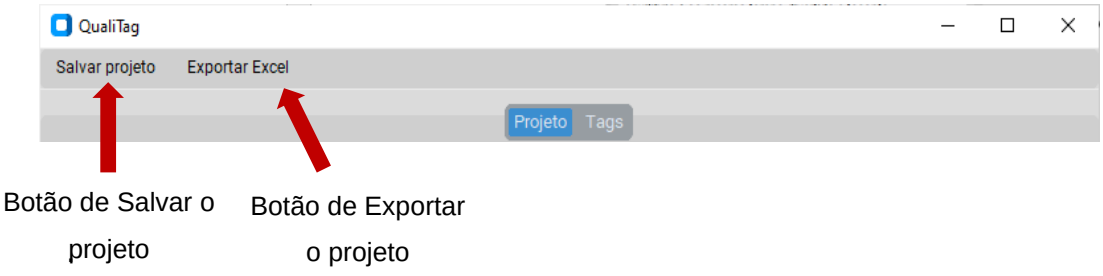
Figura 33 - Procedimento para codificação



Fonte: Elaborado pelos autores.

Por fim, você pode salvar ou exportar os projetos nos formatos indicados no menu superior, indicado na Figura 34. **Importante:** sempre lembre-se se salvar o projeto antes de fechar.

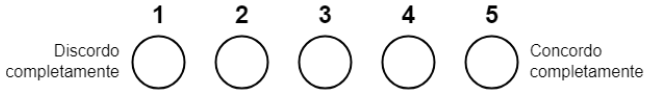
Figura 34 - Opções de salvar e exportar o projeto



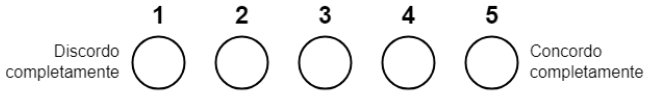
Fonte: Elaborado pelos autores.

ANEXO A - QUESTIONÁRIO *SYSTEM USABILITY SCALE*

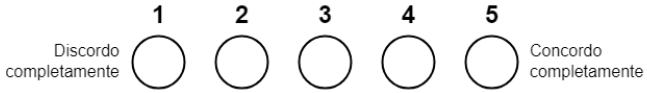
1. Eu acho que gostaria de usar esse sistema com frequência.



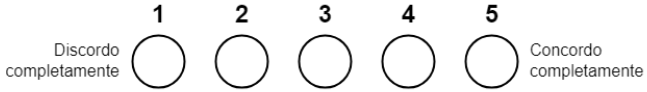
2. Eu acho o sistema desnecessariamente complexo.



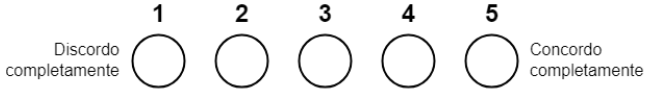
3. Eu achei o sistema fácil de usar.



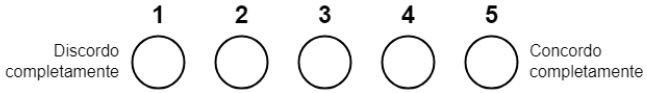
4. Eu acho que precisaria de ajuda de uma pessoa com conhecimentos técnicos para usar o sistema.



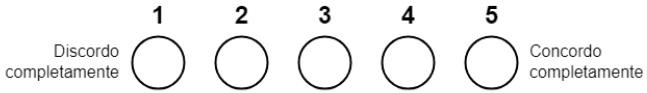
5. Eu acho que as várias funções do sistema estão muito bem integradas.



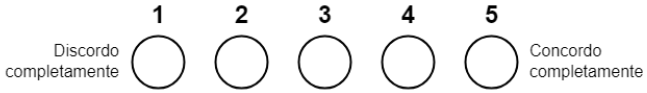
6. Eu acho que o sistema apresenta muita inconsistência.



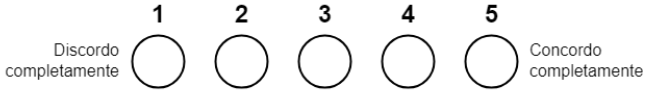
7. Eu imagino que as pessoas aprenderão como usar esse sistema rapidamente.



8. Eu achei o sistema atrapalhado de usar.



9. Eu me senti confiante ao usar o sistema.



10. Eu precisei aprender várias coisas novas antes de conseguir usar o sistema.

